

Modul 306

Projektstudie TD-Shoot

Status	Realisiert
Projektname	TD-Shoot
Projektleiter	Stefanie Gerber
Auftraggeber	Michal Durik
Autoren	Stefanie Gerber, Christophe Grädel, Benjamin Phengrasamy
Verteiler	St.G, Ch.G, B.P, S.-S, M.D

Änderungskontrolle, Prüfung, Genehmigung

Version	Datum	Beschreibung, Bemerkung	Name oder Rolle
1.0	03.03.2026	Erst Entwurf	Stefanie Gerber
1.1	10.03.2026	Ergänzung Ergebnisse M1	Stefanie Gerber
1.2	10.03.2026	Testkonzepte erstellen	Christophe Grädel
1.3	11.03.2026	Ergänzungen, Formatierungen	Stefanie Gerber
1.4	17.03.2026	Ergänzungen, Arbeitsjournale	Stefanie Gerber
1.5	24.03.2026	Finalisierungen, Arbeitsjournale	Stefanie Gerber/ Christophe Grädel
2.0	25.03.2026	Anhänge, Abschluss, finale Version	Alle

Definitionen und Abkürzungen

Begriff / Abkürzung	Bedeutung
St.G/ PL	Stefanie Gerber (Projektleitung)
Ch.G/ DEV, QA	Christophe Grädel (Development, Qualitätssicherungsaufgaben)
B.P/ BA, QA	Benjamin Phengrasamy (Business Analyst, Qualitätssicherung- Aufgaben)
S.S/ QA	Sujanthan Sunthoralingam (Qualitätssicherungsaufgaben)
M.D	Michal Durik (Auftraggeber/ Dozent)

Referenzen

Referenz	Titel, Quelle
GitHub	Repo: https://github.com/seanat0r/306_top_down_shooter
Hermes	https://www.hermes.admin.ch/en/
MS Teams	Link M306 SharePoint

Inhaltsverzeichnis

1	Arbeitsjournal	3
1.1	Initialisierung: Dienstag, 03.03.2026	3
1.2	Konzept: Dienstag, 10.03.2026	4
1.3	Realisierung Tag 1: Dienstag, 17.03.2026	5
1.4	Realisierung Tag 2: Mittwoch, 18.03.2026	6
1.5	Einführung Tag 1: Dienstag, 24.03.2026	7
1.6	Einführung Tag 2: Mittwoch, 25.03.2026	8
2	Situationsanalyse	9
2.1	Ausgangslage	9
2.2	Stärken	9
2.3	Schwächen	10
3	Ziele	10
3.1	Rahmenbedingungen	11
3.2	Abgrenzung	12
4	Liste der Stakeholder	13
5	Projektstrukturplan	14
6	Risikoanalyse	15
6.1	Risikograph	17
7	Zeitplanung mit Meilensteinen	18
8	Anforderungen	18
9	Beschreibung Spiel	20
9.1	Game Loop	20
9.2	Entity	20
9.2.1	Spieler	20
9.2.2	Gegner	20
9.2.3	Projektile (Schüsse)	20
9.2.4	Umgebung	20
9.2.4.1	Level 1	20
9.2.4.2	Level 2	21
9.3	Benutzeroberfläche (Details)	21
9.4	Software-Architektur	21
10	Testkonzept	22
10.1	Unit/ Gameplaytests	22
10.1.1	Player	22
10.1.2	Enemy	22
10.1.3	Projectile	22
10.2	Abnahmetests	23
10.3	Testrahmen	23
10.4.1	Bug 1	23
10.4.2	Bug 2	23
10.4.3	Fehlerklassen	24
10.5	Testfallbeschreibungen	24
14	Schlussbericht	32
14.1	Management Summary	32
14.2	Zielerreichung	33
14.2.1	Ziel Z1: Spielbares MVP innerhalb von sechs Kurstagen	33
14.2.2	Ziel Z2: Strukturierte Projektplanung nach HERMES 5	34
14.2.3	Ziel Z3: Aufbau von Praxiwissen in Game-Development und Python	35
14.2.4	Ziel Z4: Hohe Nachvollziehbarkeit durch GitHub-Nutzung	35
14.3	Anforderungserfüllung	36
14.4	Unit-Tests / Gameplay-Tests	37
14.5	Abnahmetests	37
15	Reflexionen des Projektteams	38
15.1	Projektleitung (Stefanie Gerber)	38
15.2	Development (Christophe Grädel)	38
15.3	Business Analyse (Benjamin Phengrasamy)	39
15.4	Quality Assurance (Sujanthan Suntheralingam)	39
15.5	Gesamtfazit und Empfehlungen	39
15.5.1	Kritische Erfolgsfaktoren	40
15.5.2	Verbesserungspotenziale	40
15.5.3	Ausblick: Potenzielle Weiterentwicklung	40
16	Schlusswort	41
16	Abbildungsverzeichnis	42
17	Anhänge	43

1 Arbeitsjournal

Das Arbeitsjournal dient der Nachvollziehbarkeit der vom Team ausgeführten Arbeiten und wird in die Bewertung mit einbezogen.

1.1 Initialisierung: Dienstag, 03.03.2026

Tätigkeiten	Person	Aufwand geplant (Std)	Aufwand effektiv (Std)
<i>Projekt Initialisierung M1</i> Team arbeitet an Arbeitspaketen 1.0- 1.5	alle	4x 3.0	4x 3.0
<i>Meeting Stakeholder</i> M1 abschliessen	alle	4x 0.25	4x 0.3
Total:			4x 3.3
Tagesablauf			
Alles verlief wie geplant, Wissenmissstände wurden eruiert.			
Hilfestellungen			
Zusatzaufgabe: GitHub Bedingung, How to «commit» Anleitung von Dev.			
Reflexion			
PL: Klarer, direkter kommunizieren. Initiative Einzelner könnte höher sein.			
Nächste Schritte			
10.03.26 Konzept M2			

1.2 Konzept: Dienstag, 10.03.2026

Tätigkeiten	Person	Aufwand geplant (Std)	Aufwand effektiv (Std)
<i>Projekt Konzept M2</i> Team arbeitet an Arbeitspaketen 2.0- 2.5	alle	4x 3.0	4x 3.0
<i>Meeting Stakeholder</i> M2 Abschliessen, Dokumenten einsicht	alle	4x 0.2	4x 0.2
<i>Abschluss Meeting</i> Ganze Klasse	alle	4x 0.2	4x 0.2
Total:			4x 3.4
Tagesablauf			
Programm wurde durchgeführt, Missstände eruiert und von PL/Dev übernommen und durchgeführt.			
Hilfestellungen			
PL übernahm Zeitplan von BA			
PL half QA bei Tasks			
Reflexion			
PL: Wissens Missstände im Team führen oft zu Komplikationen, welche in mehr aufwand enden.			
Nächste Schritte			
Vorgabe für BA Zeitplan bis 13.03 fertig, 17.03.26 erster Tag Realisierung M3			

1.3 Realisierung Tag 1: Dienstag, 17.03.2026

Tätigkeiten	Person	Aufwand geplant (Std)	Aufwand effektiv (Std)
<i>Projekt Realisierung M3</i> Team arbeitet an Arbeitspaketen 3.0- 3.6	alle	4x 3.0	4x 3.0
<i>Meeting Stakeholder</i> Projektfortschritt	alle	4x 0.25	4x 0.25
Total:			4x 3.3
Tagesablauf			
Teaminterne Differenzen wurden mit Stakeholder besprochen, weiteres Vorgehen definiert. Restlicher Abend verlief wie geplant. Team ist auf gleichem Stand.			
Hilfestellungen			
PL erstellt Taskliste für QA			
Reflexion			
DEV: Die Programmierung verlief planmässig und strukturiert. Ich konnte mir schnell einen klaren Überblick über die notwendigen Arbeitsschritte verschaffen. Durch das sofortige Testen liessen sich kleinere Fehler direkt im Entstehungsprozess beheben.			
Nächste Schritte			
Realisierung Tag 2: Mittwoch, 18.03.2026			

1.4 Realisierung Tag 2: Mittwoch, 18.03.2026

Tätigkeiten	Person	Aufwand geplant (Std)	Aufwand effektiv (Std)
<i>Projekt Realisierung M3</i> Team arbeitet an Arbeitspaketen 3.0- 3.6	alle	4x 3.0	4x 3.0
<i>Meeting Stakeholder</i> <i>Projekt Realisierung</i>		4x 0.25	4x 0.25
Total:			4x 3.3
Tagesablauf			
Team Arbeitet selbstständig an Tasks			
Hilfestellungen			
Reflexion			
BA: Zu Beginn war ich mit der Umsetzung meines Terminplan erfreut und konnte mit Teammitglieder abklären, aber das "pushen" über GitBash-Terminal in den GitHub war meine Herausforderung. Mit Hilfe von meinen Teamkollegen und Teamleader konnte ich lernen wie ich es umsetze und was ich beachten soll.			
Nächste Schritte			
Einführung Tag 1: Dienstag, 24.03.2026			

1.5 Einführung Tag 1: Dienstag, 24.03.2026

Tätigkeiten	Person	Aufwand geplant (Std)	Aufwand effektiv (Std)
<i>Projekt Einführung M4</i> Team arbeitet an Arbeitspaketen 4.0- 4.6	alle	3x3.3	3x4.0
<i>Meeting Stakeholder</i> Projekt Implementation	alle	3x0.2	3x0.3
Total:			3x4.3
Tagesablauf			
Erarbeitung von Anhängen und Schlussberichten			
Hilfestellungen			
QA-Tasks aufgeteilt unter PL, BA, DEV			
Reflexion			
QA verlässt unser Team, aus persönlichen Gründen.			
Nächste Schritte			
Einführung Tag 2			

1.6 Einführung Tag 2: Mittwoch, 25.03.2026

Tätigkeiten	Person	Aufwand geplant (Std)	Aufwand effektiv (Std)
<i>Proekt abschluss M4</i> Finalisierung Studie, aufräumen Github	PL, DEV	2x3.0	4x3
<i>PowerPoint Präsentation</i> Erarbeitung und Finalisierung	BA	1x 3.0	
<i>Team Meeting «Schlusswort»</i>	alle	3x 0.5	
Total:			
Tagesablauf			
Finalisierungs arbeiten, Abschlüsse, Organisatorisches			
Hilfestellungen			
PL: Finalisierung Studie DEV: Aufräumen Github Repo BA: Finalisierung PPP			
Reflexion			
Teamfazit: Das Projekt lief sehr gut. Alle Anforderungen wurden erreicht. Das Team hat bis auf kleine Umstände gut zusammengearbeitet.			
Nächste Schritte			
31.03.2026 PP/Verkauf an Stakeholder mit Publikum			

2 Situationsanalyse

Im Rahmen des Moduls 306A „IT Projekt realisieren“ sollen die Lernenden in Teams ein vollständiges, aber bewusst klein gehaltenes IT-Projekt nach der Methode HERMES 5 durchführen. Die Klasse übernimmt dabei die Rolle fiktiver Start-ups im Bereich Spieleentwicklung und entwickelt innerhalb von sechs Kurstagen ein lauffähiges Spielprodukt inklusive Projektdokumentation. Der Dozent tritt als Auftraggeber auf, definiert die Rahmenbedingungen und bewertet sowohl das Spiel als auch die Einhaltung der HERMES-Vorgaben über mehrere Meilensteine hinweg. Unser Team hat sich entschieden, ein Top-Down-Shooter-Spiel als ausführbare Python-Anwendung (EXE) zu realisieren und alle geforderten HERMES-Ergebnisse im GitHub-Repository zu führen. Die vorliegende Projektstudie analysiert die Ausgangslage, zeigt Ziele und Varianten auf und dient als Entscheidungsgrundlage für die Freigabe des Projekts.

2.1 Ausgangslage

Wir sind ein vierköpfiges Team im Modul 306A und stehen vor der Aufgabe, in einem begrenzten Zeitraum ein funktionsfähiges, prüfbares Spiel zu konzipieren, umzusetzen und einzuführen. Das Projekt wird als Kleinprojekt nach HERMES 5 mit den Phasen Initialisierung, Konzept, Realisierung und Einführung durchgeführt, wobei alle relevanten Ergebnisse im GitHub-Repository strukturiert abgelegt werden müssen. Der Dozent bewertet den Projekterfolg anhand der HERMES-Konformität, der Qualität des Produkts und der Professionalität der Zusammenarbeit, sodass neben der technischen Umsetzung auch Planung, Steuerung und Dokumentation im Fokus stehen. Für die Initialisierung wurde bereits ein Projektinitialisierungsantrag (PIA) erstellt, der Projektname, Team, Rollen, grobe Ziele, Aufwandsschätzung und Risiken für die Phase Initialisierung festhält. Aufbauend auf diesen Grundlagen präzisiert die Projektstudie nun Anforderungen, Lösungsvarianten, Planung und Empfehlung für das Gesamtprojekt TD-Shooter

2.2 Stärken

In der heutigen Situation profitieren wir von einem klar strukturierten Ausbildungsrahmen im Modul 306A mit definierten HERMES-Phasen, Meilensteinen und Bewertungskriterien, was uns Orientierung und Planungssicherheit gibt. Das Projektteam besteht aus vier motivierten Mitgliedern mit unterschiedlichen Stärken in Bereichen wie Programmierung, Analyse, Qualitätssicherung und Zusammenarbeit, wodurch Aufgaben gezielt verteilt und Kompetenzen optimal genutzt werden können. Zudem stehen uns etablierte technische Ressourcen wie ein gemeinsames GitHub-Repository, eine einheitliche Python-Entwicklungsumgebung sowie digitale Kommunikationskanäle zur Verfügung, die eine transparente Versionierung, Nachvollziehbarkeit und effiziente Zusammenarbeit ermöglichen. Diese vorhandenen Stärken: klarer methodischer Rahmen, eingespieltes Team und funktionierende Tool-Landschaft, sollen im Projekt unbedingt erhalten und bewusst weiter genutzt werden, damit sie durch neue Lösungsansätze oder Veränderungen nicht verloren gehen.

2.3 Schwächen

S1: Begrenzte Projektdauer und hoher Zeitdruck

Die zur Verfügung stehenden sechs Kurstage sind sehr knapp bemessen, um ein vollständiges Spiel mit allen HERMES-Ergebnissen zu planen, umzusetzen, zu testen und einzuführen. Das Projekt muss daher helfen, Prioritäten konsequent zu setzen und ein realistisches MVP zu definieren, damit Ziele innerhalb des Zeitrahmens erreichbar bleiben.

S2: Geringe Erfahrung mit Game-Development und EXE-Build

Das Team hat im Vergleich zu klassischen Schulprojekten wenig praktische Erfahrung mit Spielelogik, Game-Design und der Erstellung von ausführbaren Python-Anwendungen. Durch das Projekt sollen strukturierte Konzepte, frühe technische Tests und klare Architekturentscheidungen sicherstellen, dass technische Hürden früh erkannt und beherrschbar werden

S3: Koordination im Homeoffice und verteilte Arbeitsplätze

Die Zusammenarbeit findet überwiegend im Homeoffice statt, was Absprachen, spontane Abstimmungen und gemeinsame Tests erschwert. Das Projekt soll mit klar definierten Kommunikationskanälen, Meeting-Rhythmen und GitHub-Workflows (Issues, PRs, Reviews) die Teamkoordination verbessern und Transparenz erhöhen.

S4: Risiko unklarer Ziele und Scope

Ohne saubere Abstimmung besteht die Gefahr, dass der Umfang des Spiels zu gross wird oder sich während des Projekts unkontrolliert verändert. Mit der Projektstudie und den HERMES-Dokumenten sollen Ziele, Scope und Änderungen sauber dokumentiert und gesteuert werden, um Überlastung und Missverständnisse zu vermeiden.

3 Ziele

Z1: Spielbares MVP des TD-Shooters innerhalb von sechs Kurstagen liefern

Es wird ein lauffähiges, klar abgegrenztes Minimal-Viable-Product (ein Level, Basis-Steuerung, Punkte/Feedback) des Top-Down-Shooters entwickelt und veröffentlicht. Dieses Ziel entschärft insbesondere die Schwäche S1 (Zeitdruck) und S4 (unklarer Scope), weil der Umfang bewusst klein gehalten und schriftlich fixiert wird.

Z2: Strukturierte Projektplanung und Steuerung nach HERMES 5

Das Projekt wird konsequent entlang der HERMES-Phasen Initialisierung, Konzept, Realisierung und Einführung mit den geforderten Dokumenten und Meilensteinen geplant und gesteuert. Damit werden die Schwächen S1 (Zeitdruck) und S3 (Koordination im Homeoffice) adressiert, da Aufgaben, Termine, Rollen und Kommunikation klar geregelt sind.

Z3: Aufbau von Praxiswissen in Game-Development und Python-EXE

Das Team sammelt konkrete Erfahrungen in Game-Design, Spielelogik und im Erstellen einer ausführbaren Python-Anwendung inklusive einfacher Build-/Release-Steps. Dieses Ziel zielt direkt auf die Schwäche S2 (geringe Game-Dev-Erfahrung), indem bewusst technische Lernziele und frühe Prototypen eingeplant werden.

Z4: Hohe Nachvollziehbarkeit und Qualität durch GitHub-Nutzung

Alle Artefakte (Code, Dokumente, Tests, Protokolle) werden versioniert im GitHub-Repository geführt, mit Issues, Branches, Pull-Requests und mindestens einem Review pro Merge. So werden die Schwächen S2 und S3 reduziert, weil Zusammenarbeit, Qualitätssicherung und Transparenz im verteilten Setting verbessert werden.

3.1 Rahmenbedingungen

Administrativ

Das Projekt findet im Rahmen des Moduls 306A „IT Projekt realisieren“ als praktische Prüfung statt und wird vom Dozenten als Auftraggeber bewertet; alle Nachweise müssen im GitHub-Repository abgelegt werden. Die Bewertung orientiert sich an HERMES-Konformität, Produktqualität und Professionalität der Zusammenarbeit.

Organisatorisch

Das Projektteam besteht aus vier Personen (Stefanie, Christophe, Benjamin, ~~Sujanthan~~) mit definierten Rollen nach HERMES (PL, BA, DEV, QA), wobei Doppelfunktionen möglich sind. Die Zusammenarbeit erfolgt hauptsächlich im Unterricht und im Homeoffice über vereinbarte Online-Kanäle (z.B. Teams/Discord/E-Mail) sowie das gemeinsame GitHub-Repository.

Zeitlich

Die Gesamtdauer beträgt sechs Kurstage, aufgeteilt auf die Phasen Initialisierung (Tag 1), Konzept (Tag 2), Realisierung (Tag 3 - 4) und Einführung (Tag 5 - 6) mit den Meilensteinen M1 - M4. Die Initialisierung findet am 03.03.2026 in einem klar begrenzten Unterrichtsblock (ca. 18:00 - 21:15) statt; die weiteren Phasen folgen gemäss Kursplanung

Methodisch/technisch

Das Projekt wird als HERMES-Kleinprojekt mit den Phasen Initialisierung, Konzept, Realisierung und Einführung durchgeführt, alle geforderten HERMES-Ergebnisse (PIA, PMP, GDD, Testdokus etc.) werden im Ordner /docs geführt. Technische Basis sind eine Python-Entwicklungsumgebung auf den privaten Arbeitsplätzen, das gemeinsame GitHub-Repository mit klarer Struktur (Code-Ordner, Dokumente, Issues, Branches) sowie als Zielprodukt eine ausführbare Python-Anwendung (EXE) des TD-Shooters.

3.2 Abgrenzung

A1: Funktionsumfang des Spiels

Das Projekt beschränkt sich auf ein einfaches Singleplayer-Top-Down-Shooter-Spiel mit einem klar definierten MVP (z.B. ein Level, einfache Gegner, grundlegende Steuerung, Punktesystem). Nicht umgesetzt werden komplexe Features wie Multiplayer, Online-Highscore, umfangreiche Level-Systeme, Story-Modus oder persistente Speicherstände.

A2: Technische Plattform und Performance

Zielpattform ist eine lokale ausführbare Python-Anwendung (EXE); Portierungen auf mobile Geräte, Web-Browser oder andere Betriebssysteme ausserhalb des Kurs-Setups sind nicht Bestandteil des Projekts. Leistungsoptimierungen für sehr schwache Hardware oder Spezialeffekte (aufwändige 3D-Grafik, Shader, Sound-Engine mit vielen Effekten) werden nur rudimentär oder gar nicht behandelt.

A3: Umfang der Projektorganisation

Das Projekt bildet ein HERMES-Kleinprojekt im schulischen Kontext ab; Themen wie Beschaffung, Budgetverwaltung mit realen Kosten, vertragliche Aspekte oder langjährige Wartung und Betrieb sind explizit ausgeschlossen. Auch die Weiterentwicklung nach dem Kurs (z.B. neue Versionen, Community-Support) ist nicht Teil dieses Projekts, sondern könnte allenfalls als separates Folgeprojekt betrachtet werden.

A4: Inhaltliche und rechtliche Aspekte

Es werden ausschliesslich selbst erstellte oder frei nutzbare Grafiken/Sounds in einfachem Stil verwendet; professionelle Grafik-/Sound-Produktion ist nicht Ziel des Projekts. Eine vertiefte rechtliche Analyse (Lizenzen, Datenschutz, Monetarisierung) erfolgt nur soweit nötig für den Schulkontext und wird darüber hinaus nicht abgedeckt.

4 Liste der Stakeholder

Kürzel	Stakeholder / Gruppe	Rolle / Kategorie	Interesse am Projekt
ST1	Dozent / Auftraggeber (Michal Durik)	Auftraggeber, Bewertung, „Geschäftsleitung“ Schule	Erfolgreicher Projektverlauf, HERMES-Konformität, prüfbares Spiel.
ST2	Projektteam (Stefanie, Christophe, Benjamin, Sujan than)	Projektorganisation / Umsetzungsteam	Bestehensrelevante Leistung, Lernen von PM und Game-Dev, gute Zusammenarbeit.
ST3	Mitschüler:innen / Klasse	Indirekt Betroffene, potenzielle Test-User	Vergleichbare Projekterfahrung, Austausch, evtl. Spiel testen.
ST4	Schule / Bildungsgang (z.B. ICT-Ausbildung)	Stammorganisation / Träger	Nachweis, dass Modulziele (Projektarbeit, HERMES, GitHub) erreicht werden.
ST5	Fiktive End-User (Spieler des TD-Shooters)	Kunden / Nutzer	Einfach spielbares, verständliches und stabiles Game.
ST6	System-/Infrastrukturverantwortliche Schule	IT-Umgebung / Infrastruktur	Sicherstellung, dass verwendete Tools (GitHub, Python) im Schulkontext funktionieren

Informiert werden müssen mindestens der Dozent als Auftraggeber (laufend über Meilensteine und Repo-Zugriff) sowie das Projektteam intern über Stand, Aufgaben und Entscheidungen. Weitere Stakeholder wie Mitschüler:innen oder die Schule werden hauptsächlich über Präsentationen, Reviews und das öffentlich einsehbare GitHub-Repository einbezogen.

5 Projektstrukturplan

Phasenübergreifende Hauptstruktur (HERMES-basiert):

1. Initialisierung
 - AP1.1 Team bilden, Rollen definieren - PL, gesamtes Team.
 - AP1.2 Spielidee wählen, MVP und Scope grob definieren - PL, BA.
 - AP1.3 Stakeholder- und Risikoanalyse erstellen - PL.
 - AP1.4 GitHub-Repo und Grundstruktur (/src, /docs) einrichten - DEV.
 - AP1.5 PIA erstellen und Meilenstein M1 vorbereiten - PL, BA
2. Konzept
 - AP2.1 Anforderungen (funktional/nicht-funktional) definieren - BA, QA.
 - AP2.2 Game-Design / Lösungskonzept (Spiellogik, UI-Skizzen, Architektur) - DEV, BA.
 - AP2.3 Projektmanagementplan (Termine, Ressourcen, Qualität) - PL.
 - AP2.4 Teststrategie und grobe Testfälle definieren - QA.
 - AP2.5 Meilenstein M2 vorbereiten (Präsentation Konzept) - PL, gesamtes Team
3. Realisierung
 - AP3.1 Basis-Spielgerüst implementieren (Player, Steuerung, Loop) - DEV.
 - AP3.2 Gegner, Kollision, Punktesystem implementieren - DEV.
 - AP3.3 Issues, Branches, PR-Reviews im GitHub führen - PL, DEV, QA.
 - AP3.4 Tests durchführen und dokumentieren (Unit-/Gameplay-Tests) - QA, DEV.
 - AP3.5 Benutzerhandbuch Entwurf erstellen (README.md) - BA, QA, DEV.
 - AP3.6 Statusbericht und Vorbereitung M3 - PL
4. Einführung
 - AP4.1 Abnahmetests gegen Anforderungen durchführen und protokollieren - QA
 - AP4.2 EXE-Build/Release vorbereiten (Paket, Startanleitung) - DEV.
 - AP4.3 Abschlussdokumentation (Schlussbericht, Lessons Learned) - PL, gesamtes Team.
 - AP4.4 Abschlusspräsentation vorbereiten und durchführen - gesamtes Team.
 - AP4.5 Repository abschliessen/archivieren (Tags, Releases, Aufräumen) - PL, DEV.
 - AP4.6 Erstellung Powerpointpräsentation - BA, QA

6 Risikoanalyse

Risikobeschreibung	Auswirkung	Auswirkung				Nach Massnahme			
		W	R	Risiko	Handlungsweise (Massnahme)	W	S	Risiko	Handlungsweise
R1: Zeit reicht für Initialisierung / Projektphasen nicht aus	PIA, Konzepte oder Implementierung sind unvollständig, Meilensteine können nicht erreicht werden, Gefahr von Punkteabzug bis hin zum Nichtbestehen	W4	S3	hoch	Klare Agenda und Timeboxing je Sitzung, Priorisierung auf MVP, offene Punkte als To-dos dokumentieren, frühzeitige Abstimmung mit Auftraggeber bei Verzögerungen.	W3	S2	mittel	Laufende Kontrolle des Terminplans, bei Bedarf Scope weiter reduzieren
R2: Unklare Ziele / Scope des Spiels	Überladener Funktionsumfang, Team verzettelt sich, Anforderungen werden nicht erfüllt, Qualität leidet, deutlicher Notenabzug möglich	W4	S3	hoch	MVP schriftlich definieren (1 Level, Basis-Features), Ziele und Nicht-Ziele im Projektstudien-Dokument festhalten, PL entscheidet bei Unklarheiten, Änderungen nur via Change-Request	W2	S2	gering - mittel	Regelmässige Scope-Reviews, neue Ideen in Backlog für spätere Versionen verschieben
R3: Technische Hürden mit Python / EXE-Build / GitHub	Verzögerter Start in Konzept/Realisierung, Build schlägt am Ende fehl, Punkteabzug bei Produktqualität und Nachvollziehbarkeit	W3	S2	mittel	Früh Test-Repo und „Hello-World“-Projekt anlegen, minimalen Build-/Run-Prozess am Anfang prüfen, technische Issues sofort erfassen und priorisieren	W2	S2	gering - mittel	Technische Doku pflegen, bei Problemen frühzeitig Unterstützung holen, bewährte Tools/Libs nicht kurzfristig wechseln.

R4: Ausfall eines Teammitglieds	Weniger verfügbare Arbeitszeit im Team, Verzögerungen bei Implementierung oder Dokumentation, einzelne Rollen (z.B. QA oder BA) sind nicht mehr abgedeckt, was zu Qualitätsmängeln und Notenabzug führen kann.	W3	S3	mittel - hoch	Rollen und Wissen so verteilen, dass jede Schlüsselaufgabe mindestens von zwei Personen verstanden wird (Vertretungsregelung), wichtigen Code und Dokus laufend in GitHub pushen, Tasks klein schneiden und transparent in Issues führen, damit andere kurzfristig übernehmen können.	W2	S2	gering - mittel	Bei längerem Ausfall Tasks im PSP und in GitHub neu verteilen, Scope bei Bedarf mit Auftraggeber nach unten anpassen.
R5: Kommunikationsprobleme im Homeoffice	Aufgaben werden nicht oder doppelt erledigt, wichtige Informationen zu Scope, Architektur oder Tests gehen verloren, was zu Verzögerungen, Qualitätsmängeln und möglichen Notenabzügen führt.	W4	S2-S3	mittel - hoch	Feste Kommunikationskanäle (z.B. Teams/Discord) und Regeln definieren, tägliche kurze Abstimmungen durchführen, Aufgaben ausschliesslich über GitHub-Issues mit Verantwortlichen und Deadlines steuern, wichtige Entscheide in Protokollen im /docs-Ordner festhalten.	W2-W3	S2	gering - mittel	Bei erkennbaren Missverständnissen sofortiges Klärungsmeeting einberufen und Verantwortlichkeiten/Tasks aktuell im Board bzw. in Issues nachziehen.

Schadensausmaß:

S1 = führt zu keiner Abwertung

S2 = geringe Abwertung bis 1,0 Notenpunkte

S3 = hohe Abwertung über 1,0 Notenpunkte

S4 = führt zu Nichtbestehen

Eintrittswahrscheinlichkeit:

W1 = unvorstellbar

W2 = unwahrscheinlich

W3 = eher vorstellbar

W4 = wahrscheinlich

W5 = sehr wahrscheinlich

W6 = Eintreffen sehr hoch

6.1 Risikomatrix

Eintrittswahrscheinlichkeit	W5			
	W4			
	W3	R1, R5		
	W2	R2, R3, R4		
	W1, S1	S2	S3	S4
Schadensausmass				

Zur Einstufung der Projektrisiken verwenden wir die dargestellte Risikomatrix, der die Kombination aus Eintrittswahrscheinlichkeit (W1 – W5) und Schadensausmass (S1 - S4) in eine Ampellogik (grün/gelb/rot) überführt. Jedes Risiko wird zunächst mit W und S bewertet und dann in der Risikomatrix verortet, sodass sich die qualitative Einstufung „tief“ (grün), „mittel“ (gelb) oder „hoch“ (rot) ergibt. Nach Definition der Massnahmen werden W* und S* erneut beurteilt, um das verbliebene Restrisiko transparent zu machen.

7 Zeitplanung mit Meilensteinen

Phase	Start	Ende	Dauer (Tage)	Meilenstein	Beschreibung Meilenstein
Initialisierung	03.03.2026	03.03.2026	1	M1	Phasenfreigabe Initialisierung: PIA, Stakeholderliste, Risikoliste vorhanden, Repo eingerichtet, Freigabe durch Auftraggeber.
Konzept	10.03.2026	10.03.2026	1	M2	Phasenfreigabe Konzept: PMP, GDD/Lösungskonzept, Teststrategie vorhanden, Präsentation beim Auftraggeber, Freigabe für Umsetzung.
Realisierung	17.03.2026	24.03.2026	2	M3	Zwischenabnahme Realisierung: lauffähiger Zwischenstand, Testprotokolle, Statusbericht, offene Risiken/Changes transparent.
Einführung	24.03.2026	24.03.2026	2	M4	Schlussabnahme & Präsentation: Abnahmeprotokoll, EXE-Release, Schlussbericht, Lessons Learned, finale Bewertung

8 Anforderungen

Nr.	Anforderung	Typ	Abgedeckte Ziele:
A1	Das Spiel bietet mindestens ein vollständiges Level mit klar definiertem Start, laufendem Spielablauf und Spielende (Win/Lose-Bedingung)	funktional	Z1
A2	Der Spieler kann die Figur über Tastatur/Maus (z.B. WASD + Schiessen) präzise steuern; Steuerung wird im Spiel erklärt.	funktional	Z1
A3	Es gibt mindestens einen Gegnertyp, der sich bewegt und den Spieler schädigen kann; Treffer und Lebenspunkte (Player) werden visuell angezeigt.	funktional	Z1

A4	Das Spiel zeigt einen Punktestand bzw. Score an und setzt diesen beim Neustart korrekt zurück	funktional	Z1
A5	Das Spiel läuft stabil ohne Abstürze in der Zielumgebung (Schulrechner / Homeoffice-Setup) und reagiert flüssig.	nicht-funktional	Z1, Z3
A6	Das Spiel wird als ausführbare Python-Anwendung (EXE) bereitgestellt, inkl. kurzer Startanleitung. (README.md)	nicht-funktional	Z1, Z3
A7	Alle Anforderungen und das Game-Design sind im Game Design Document (GDD) dokumentiert und im /docs-Ordner versioniert.	nicht-funktional	Z2, Z4
A8	Für zentrale Funktionen existieren Testfälle und Testprotokolle; Abnahmetests gegen die Anforderungen werden dokumentiert.	nicht-funktional	Z2, Z4
A9	Das Projekt wird gemäss HERMES-Phasen mit den geforderten Dokumenten (PIA, PMP, GDD, Statusberichte, Schlussbericht) geführt.	nicht-funktional	Z2
A10	Das <u>GitHub</u> -Repository erfüllt die Vorgaben (README, /docs, /src, Issues, Branches, PR-Reviews, mindestens ein Release-Tag).	nicht-funktional	Z2, Z4

9 Beschreibung Spiel

Das Projekt „TD-Shooter“ ist ein einfaches 2D-Top-Down-Shooter-Spiel, in dem der Spieler eine Figur aus der Vogelperspektive über ein Spielfeld steuert und gegen auftauchende Gegner antritt. Gesteuert wird die Spielfigur mit der Tastatur (z.B. WASD für Bewegung, und Maus zum Schiessen), während Gegner den Spieler verfolgen oder beschiessen und bei Kollision bzw. Treffern Lebenspunkte abziehen. Durch das Ausschalten von Gegnern erhält der Spieler Punkte; bei Erreichen einer definierten Punktzahl oder nach einer bestimmten Überlebenszeit gilt das Level als gewonnen, bei Verlust aller Lebenspunkte als verloren.

Technisch wird das Spiel als 2D-Spiel mit einer Python-Bibliothek (Pygame) umgesetzt und als ausführbare EXE für die Zielumgebung bereitgestellt, sodass es ohne Entwicklungsumgebung gestartet werden kann. Grafiken und Sounds sind bewusst einfach gehalten, um den Fokus auf eine stabile, gut steuerbare Spiellogik und die Umsetzung der Projekt- und Qualitätsanforderungen im Rahmen der sechs Kurstage zu legen.

9.1 Game Loop

Der Kern des Spiels besteht aus dem Überlebenskampf gegen kontinuierlich spawnende Gegnerwellen.

- **Progression:** Durch das Eliminieren von Gegnern erhält der Spieler Punkte (+1 Score). Das Ziel ist das Erreichen von Level 2 und ein möglichst hoher Highscore.
- **Zustandssteuerung (Pause):** Das Spiel nutzt eine State-Machine. Über die ESC-Taste kann das Spiel jederzeit pausiert werden. Dabei wird die Update-Logik (Bewegung, Spawning, Timer) eingefroren, während das aktuelle Spielfeld im Hintergrund sichtbar bleibt.

9.2 Entity

9.2.1 Spieler

- **Leben:** Maximal 3 Lebenspunkte.
- **Munition:** Begrenztes Magazin mit manuellem/automatischem Reload (R-Taste). Als Belohnung für den Fortschritt erhöht sich die maximale Magazinkapazität alle 100 Punkte um +5 Schuss.
- **Score** Akkumulierte Punkte durch besiegte Gegner.
- **Waffe:** Eine Fernkampfwaffe (Projektil Basis).

9.2.2 Gegner

- **Skalierung:** Mit steigendem Score erhöhen sich die Geschwindigkeit und die Spawnmenge und die Schwierigkeit der Gegner.
- **Leben:** Startwert beträgt 1 HP.
- **Nahkampfschaden:** Bei physischem Kontakt mit dem Spieler verliert dieser Lebenspunkte.
- **Bewegung:** Gegner verfolgen den Spieler aktiv auf der Karte.
- **Spawn-System:** Gegner erscheinen an den Kartenrändern, jedoch außerhalb eines Sicherheitsradius um den Spieler.

9.2.3 Projektile (Schüsse)

- Kleine Projektile, die bei Kollision Schaden verursachen.
- **Kollisionsverhalten**
 - **Hindernisse:** Das Projektil wird bei Kontakt mit der Umgebung zerstört.
 - **Gegner:** Das Projektil fügt dem Gegner Schaden zu und wird meist zerstört.

9.2.4 Umgebung

Die Karten werden manuell erstellt und enthalten statische Hindernisse, die weder von Spielern noch von Gegnern oder Projektilen passiert werden können.

9.2.4.1 Level 1

Wenige Hindernisse; dient als Tutorial zur Einführung in die Steuerung und Mechaniken.

9.2.4.2 Level 2

Erhöhte Komplexität durch viele Hindernisse und Sackgassen, um den Spieler strategisch zu fordern.

9.3 Benutzeroberfläche (Details)

Bei einer Munitionserhöhung oder wenn die Gegneranzahl pro Welle erhöht wird, kommt eine Benachrichtigung.

Das HUD besteht aus vier Kernelementen:

- **Lebensanzeige:** Horizontal am oberen linken Bildschirmrand.
- **Score:** Direkt unter der Lebensanzeige.
- **Munitionsanzeige:** Vertikaler Balken in der unteren rechten Ecke.
- **Spielfeld:** Zentraler Bereich für die Action.

Design-Update: Zur Verbesserung der visuellen Klarheit und Hitbox-Präzision wird der Spieler als minimalistisches Quadrat dargestellt

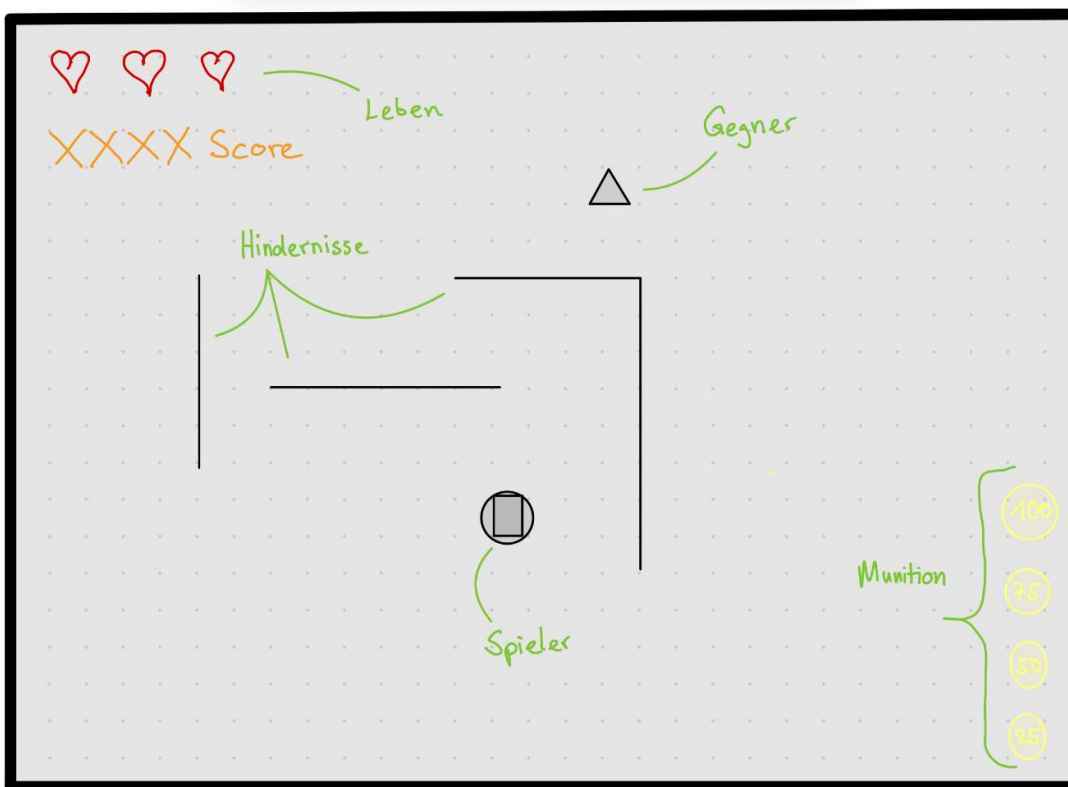


Abb. 1 TD-Shoot UI Konzept

9.4 Software-Architektur

Das Spiel folgt dem EVA-Prinzip (Eingabe, Verarbeitung, Ausgabe) und nutzt eine klassenbasierte Struktur:

- **Game-Klasse (Controller):** Zentrales Management des Main-Loops, des Event-Handlings (Input) und der Level-Übergänge.
- **Sprite-System:** Nutzung von `pygame.sprite.Sprite` für:
 - *Player:* Inklusive HP-Management und Bewegungslogik.
 - *Enemy:* Inklusive Pathfinding/KI und Schadenslogik.
 - *Projectile:* Berechnung der Flugbahn und Kollisionsabfrage.
- **Level-Manager:** Verantwortlich für das Laden der Map-Daten (Kollisionsmatrix).
- **UI-Manager:** Rendering des HUDs als Overlay über dem Spielgeschehen.
- **Config:** Alle Konstanten befinden sich dort.

10 Testkonzept

10.1 Unit/ Gameplaytests

In diesem Abschnitt wird die Verifizierung der technischen Kernlogik dokumentiert. Ziel der Unit-Tests ist es, die Funktionalität der Spiel-Objekte isoliert von der grafischen Oberfläche sicherzustellen. Alle Tests wurden mit dem Framework pytest durchgeführt.

10.1.1 Player

Für die Player-Klasse wurden die logischen Zustände und Berechnungen geprüft.

- **Testumfang:** Korrekte Reduktion der Lebenspunkte (HP) bei Treffern, Überprüfung des Sterbestatus bei 0 HP sowie die Munitionsverwaltung (Abzug pro Schuss, Nachladestatus).
- **Ergebnis:** Alle 6 Testfälle für den Player wurden erfolgreich bestanden (100% Pass-Rate).

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /home/chg/csbe/306_top_down_shooter/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/chg/csbe/306_top_down_shooter
plugins: anyio-4.12.1
collected 6 items

tests/test_player.py::test_take_damage PASSED [ 16%]
tests/test_player.py::test_player_death PASSED [ 33%]
tests/test_player.py::test_hp_not_negative PASSED [ 50%]
tests/test_player.py::test_shoot_reduces_amm0 PASSED [ 66%]
tests/test_player.py::test_shoot_fails_when_empty PASSED [ 83%]
tests/test_player.py::test_cannot_shoot_while_reloading PASSED [100%]

===== 6 passed in 0.40s =====
```

Abb. 2 TD-Shoot Unit/Gameplay Tests 1/3

10.1.2 Enemy

Die Tests für die Enemy-Klasse konzentrierten sich auf die KI-Logik und Interaktion.

- **Testumfang:** Verifizierung der Vektorberechnung zur Verfolgung des Spielers, korrekte Schadensannahme sowie die Rückgabe des Score-Werts beim Ableben des Gegners.
- **Ergebnis:** Alle 4 Testfälle für die Enemy-KI wurden erfolgreich bestanden.

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /home/chg/csbe/306_top_down_shooter/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/chg/csbe/306_top_down_shooter
plugins: anyio-4.12.1
collected 10 items / 6 deselected / 4 selected

tests/test_enemy.py::test_enemy_take_hit PASSED [ 25%]
tests/test_enemy.py::test_enemy_death PASSED [ 50%]
tests/test_enemy.py::test_enemy_movement_towards_player PASSED [ 75%]
tests/test_enemy.py::test_enemy_hitbox_follows_pos PASSED [100%]

===== 4 passed, 6 deselected in 0.29s =====
```

Abb. 3 TD-Shoot Unit/Gameplay Tests 2/3

10.1.3 Projectile

Beim Projectile wurde die mathematische Flugbahn und die Lebensdauer geprüft.

- **Testumfang:** Korrekte Normalisierung des Richtungsvektors beim Instanzieren, konstante Geschwindigkeitsbewegung pro Frame sowie die Funktionalität zum Entfernen des Objekts (kill()).
- **Ergebnis:** Alle 3 Testfälle für die Projektil-Physik wurden erfolgreich bestanden.

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /home/chg/csbe/306_top_down_shooter/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/chg/csbe/306_top_down_shooter
plugins: anyio-4.12.1
collected 13 items / 10 deselected / 3 selected

tests/test_projectile.py::test_projectile_initialization PASSED [ 33%]
tests/test_projectile.py::test_projectile_movement PASSED [ 66%]
tests/test_projectile.py::test_projectile_zero_direction PASSED [100%]

===== 3 passed, 10 deselected in 0.22s =====
```

Abb. 4 TD-Shoot Unit/Gameplay Tests 3/3

10.2 Abnahmetests

Nr	Testart	Beschreibung
F1	Startup	Startet die Anwendung.
F2	Schuss	Schuss an Gegner funktioniert.
F3	Health	Leben wird abgezogen nach einem Treffer (beidseitig).
F4	Score	Scores werden gezählt.
F5	Level	Level 2 wird erreicht.
F6	Game	Spiel wird beendet.

10.3 Testrahmen

Der Testrahmen definiert Umfang, Tiefe und organisatorische Einbettung der Tests für den TD-Shooter. Getestet werden alle zentralen Funktionen des Spiels (Bewegung und Steuerung des Spielers, Schiessen, Gegnerverhalten, Kollisionen, Punkte- und Lebensanzeige, Spielstart und -ende) sowie die Startfähigkeit der EXE-Version auf der vorgesehenen Zielumgebung.

Die Tests werden primär in der Realisierungs- und Einführungsphase durchgeführt: während der Realisierung stehen Entwickler- und Gameplay-Tests im Vordergrund, in der Einführung folgen formale Abnahmetests gegen die definierten Anforderungen. Es kommen manuelle Tests anhand von vordefinierten Testfällen zum Einsatz; bei Bedarf werden zusätzlich einfache Unit-Tests für kritische Funktionen implementiert.

Für jeden Testlauf wird im Testprotokoll festgehalten, welche Testfälle durchgeführt wurden, welches Resultat erzielt wurde (bestanden/nicht bestanden) und welche Fehler entstehen; erkannte Mängel werden als Issues in GitHub erfasst und bis zur Abnahme nachverfolgt. Der Auftraggeber erhält im Rahmen der Meilensteine M3 und M4 Einsicht in Testkonzept, Testprotokolle und die Ergebnisse der Abnahmetests, die als Grundlage für die Schlussbewertung dienen.

10.4 Testerportfolio:

10.4.1 Bug 1

Titel: Gegner Glitchen durch die Wand

Schweregrad: Mittel

Schritte zur Reproduktion: 1. Gegner an einer Ecke bringen. 2. Der Spieler so positionieren das der Gegner diagonal läuft. 3. Nach eine kurze Zeit teleportiert sich der Gegner auf die andere Seite der Wand

Erwartetes vs. Tatsächliches Ergebnis: Gegner sollte nicht über die Wand teleportieren. Der Gegner hätte eigentlich an der Wand bleiben sollen.

10.4.2 Bug 2

Titel: Neustarten hat der Score nicht zurückgesetzt

Schweregrad: Schwer

Schritte zur Reproduktion: Das Spiel immer wieder spielen, ohne dabei das Programm zu schliessen. Die Gegner behalten immer die aktuelle Geschwindigkeit des Vorherigen Versuches und kumuliert sich bei jeden nächsten Versuch.

Erwartetes vs. Tatsächliches Ergebnis: Die Gegner behielten ihre Geschwindigkeit. Obwohl die Geschwindigkeit und somit der Score zurückgesetzt hätte sein müssen.

10.4.3 Fehlerklassen

Festgestellte Fehler werden in Klassen 0-4 eingestuft

Nr.	Fehlerklassen	Beschreibung
0	Fehlerfrei	Test wurde erfolgreich ausgeführt, keine Fehler festgestellt.
1	Unwesentlicher Mangel	Kleiner Fehler ohne Einfluss auf die Spielfunktion (z.B Anzeige oder Textfehler)
2	Leichter Mangel	Funktion funktioniert, aber nicht optimal oder mit kleinen Einschränkungen
3	Schwerer Mangel	Wichtige Funktion funktioniert nicht korrekt (z.B Schaden wird nicht abgezogen)
4	Kritischer Mangel	Anwendung oder Spiel funktioniert nicht mehr bzw. Stürzt ab

10.5 Testfallbeschreibungen

ID / Bezeichnung F1: Startup	T-001	Referenz zu Anforderung & Abnahmekriterium Spiel startet korrekt und Hauptmenü bzw. Spiel wird geladen	<i>Fehlerklasse: 0</i>
Beschreibung	Der Benutzer startet das Spiel. Es wird überprüft, ob das Spiel korrekt geladen wird und spielbereit ist.		
Testvoraussetzung	Spiel ist installiert & Gerät ist eingeschaltet		
Testschritte	1. GitHub Repository aufrufen und README.md lesen. 2. Navigieren zum Ordner «games» und Download der TD-Shooter.exe. 3. Ausführen der Datei (ggf. Bestätigung der Windows-Sicherheitswarnung). 4. Prüfung: Erscheint das Hauptmenü korrekt? 5. Drücken der Enter-Taste, um den Spielmodus zu starten.		
Erwartetes Ergebnis	Das Spiel lädt ohne Absturz in das Hauptmenü. Nach Betätigen der Enter-Taste wechselt die Ansicht flüssig in das erste Level.		



Abb. 5 TD-Shoot Abnahmetests 1/11



Abb. 6 TD-Shoot Abnahmetests 2/11

ID / Bezeichnung F2: Schussfunktion	T-002	Referenz zu Anforderung & Abnahmekriterium Der Spieler muss Gegner angreifen können, indem er schießt.	<i>Fehlerklasse: 0</i>
Beschreibung	Der Spieler schießt auf einen Gegner. Es wird überprüft, ob der Schuss korrekt ausgelöst wird.		
Testvoraussetzung	Spiel läuft & Gegner befindet sich im Spielfeld		
Testschritte	1. Den Mauszeiger auf ein Ziel (rotes Dreieck) ausrichten. 2. Die linke Maustaste drücken 3. Prüfen, ob ein Projektil vom Spieler in Richtung des Cursors fliegt. 4. Den Gegner so lange beschiessen, bis seine Lebenspunkte (HP) auf null sinken.		
Erwartetes Ergebnis	Beim Klicken wird ein Projektil instanziiert, das sich linear zum Ziel bewegt. Bei Kollision mit dem Gegner erleidet dieser Schaden. Sobald die HP des Gegners 0 erreichen, wird das Objekt vom Spielfeld entfernt.		

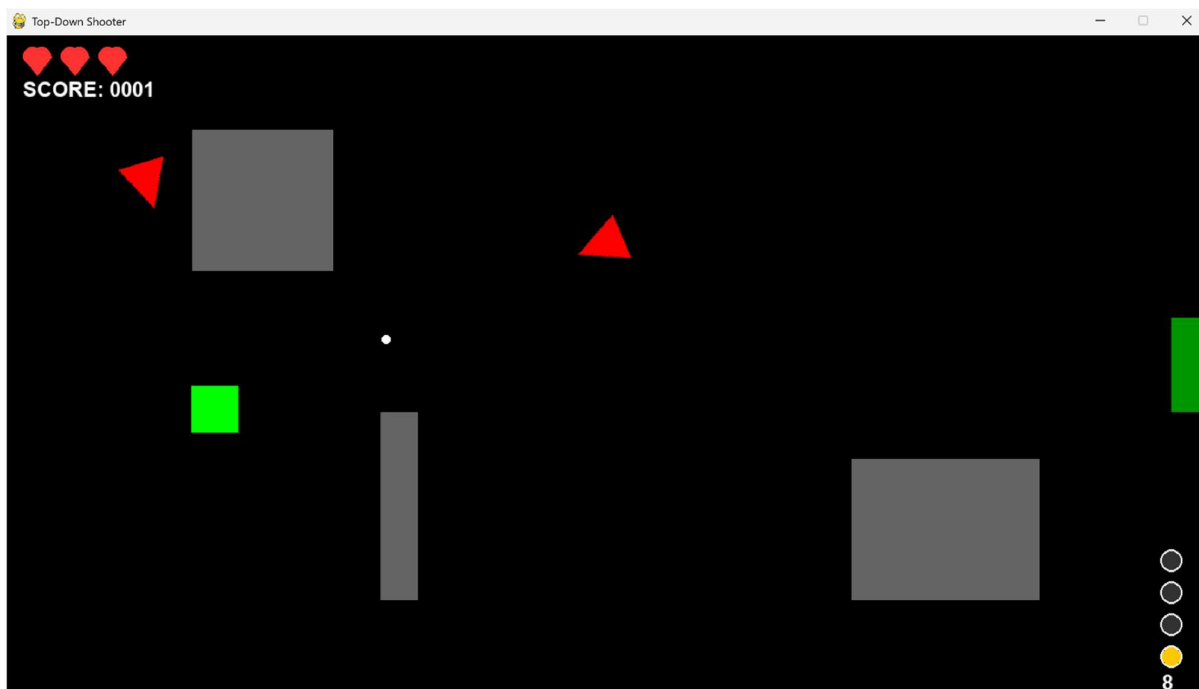


Abb. 7 TD-Shoot Abnahmetests 3/11

ID / Bezeichnung F3: Health-System	T-003	Referenz zu Anforderung & Abnahmekriterium Spieler und Gegner besitzen Lebenspunkte, die bei einem Treffer reduziert werden.	<i>Fehlerklasse: 0</i>
Beschreibung	Es wird überprüft, ob nach einem Treffer Leben vom Spieler oder Gegner abgezogen wird.		
Testvoraussetzung	Spieler und Gegner befinden sich im Spiel & Treffer erfolgt		
Testschritte	1. Den Spieler gezielt in ein Gegner steuern. 2. Die grafische Anzeige oben links beobachten. 3. Den Vorgang wiederholen, bis das letzte Herz aufgebracht ist.		
Erwartetes Ergebnis	Jede Berührung mit einem Gegner reduziert die HP um genau einen Punkt. Die Herzen im UI werden synchron dazu ausgeblendet.		



Abb. 8 TD-Shoot Abnahmetests 4/11

ID / Bezeichnung F4: Score System	T-004	Referenz zu Anforderung & Abnahmekriterium Der Spieler erhält Punkte für erfolgreiche Aktionen (z.B. Gegner besiegen).	<i>Fehlerklasse: 0</i>
Beschreibung	Der Punktestand erhöht sich korrekt nach dem Besiegen eines Gegners.		
Testvoraussetzung	Gegner wird im Spiel besiegt		
Testschritte	1. Den aktuellen Punktestand im User Interface (UI) notieren. 2. Einen Gegner anvisieren und vollständig eliminieren. 3. Die Veränderung der Punkteanzeige unmittelbar nach dem Verschwinden des Gegners prüfen.		
Erwartetes Ergebnis	Der Score-Wert erhöht sich pro besiegte Gegner um den in der Konfiguration definierten Wert (im normalen Fall: +1 Punkt). Die grafische Anzeige im UI wird ohne Verzögerung aktualisiert.		



SCORE: 0000

Abb. 9 TD-Shoot Abnahmetests 5/11



SCORE: 0001

Abb. 10 TD-Shoot Abnahmetests 6/11

ID / Bezeichnung F5: Levelsystem	T-005	Referenz zu Anforderung & Abnahmekriterium Der Spieler kann nach Erreichen bestimmter Bedingungen in das nächste Level wechseln. Nach Abschluss von Level 1 wird Level 2 korrekt geladen.	Fehlerklasse: 0
Beschreibung	Überprüfung des automatischen Szenenwechsels und der korrekten Neupositionierung des Spielers beim Erreichen des Level-Endes.		
Testvoraussetzung	Es wird überprüft, ob nach Erreichen der Bedingungen das nächste Level gestartet wird.		
Testschritte	1. Den Spieler aktiv in die grüne Fläche (Ziel-Zone) am rechten Rand steuern. 2. Den Ladevorgang von Level 2 beobachten. 3. Die Startposition des Spielers im neuen Level verifizieren.		
• Erwartetes Ergebnis	Bei Kollision mit der grünen Ziel-Zone wird Level 1 beendet und Level 2 geladen. Der Spieler wird dabei automatisch an den linken Rand (Startpunkt) des neuen Levels zurückgesetzt.		

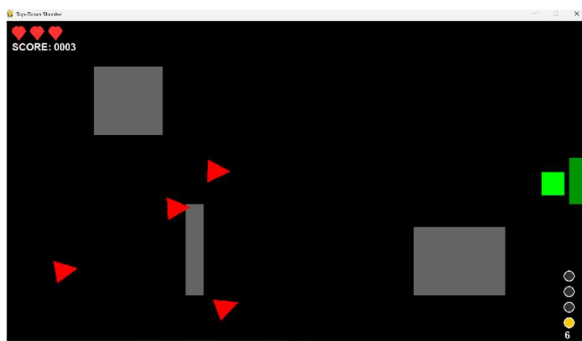


Abb. 11 TD-Shoot Abnahmetests 7/11

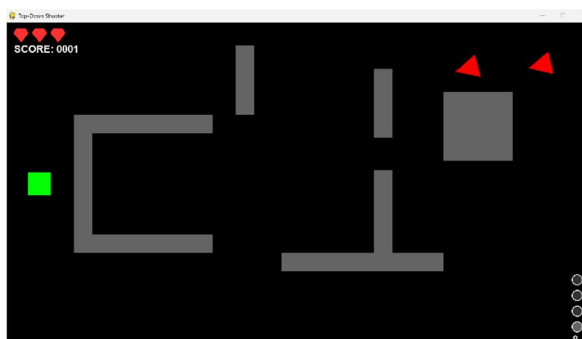


Abb. 12 TD-Shoot Abnahmetests 8/11

ID / Bezeichnung F6: Spielende	T-006	Referenz zu Anforderung & Abnahmekriterium Das Spiel muss beendet werden, wenn der Spieler keine Lebenspunkte mehr hat.	<i>Fehlerklasse: 0</i>
Beschreibung	Verifizierung des Übergangs in den Game-Over-Zustand sowie der korrekten Rücksetzung aller globalen Spielparameter bei Verlust aller HP.		
Testvoraussetzung	Spiel läuft. Spieler hat nur noch einen Lebenspunkt (1 Herz).		
Testschritte	1. Eine finale Kollision mit einem Gegner provozieren. 2. Das Erscheinen des Game-Over-Overlays prüfen. 3. Den Neustart des Spiels einleiten. 4. Kontrollieren, ob HP, Score und Gegner-Schwierigkeit (Speed/ Spawn) wieder auf den Initialwert stehen.		
Erwartetes Ergebnis	Bei 0 HP wird das Spiel unterbrochen und das Game-Over-Menü eingeblendet. Alle Variablen (Score, Leben, Gegner-Speed und Spawn-Rate) werden zwingend auf die Standardwerte der config.py zurückgesetzt.		

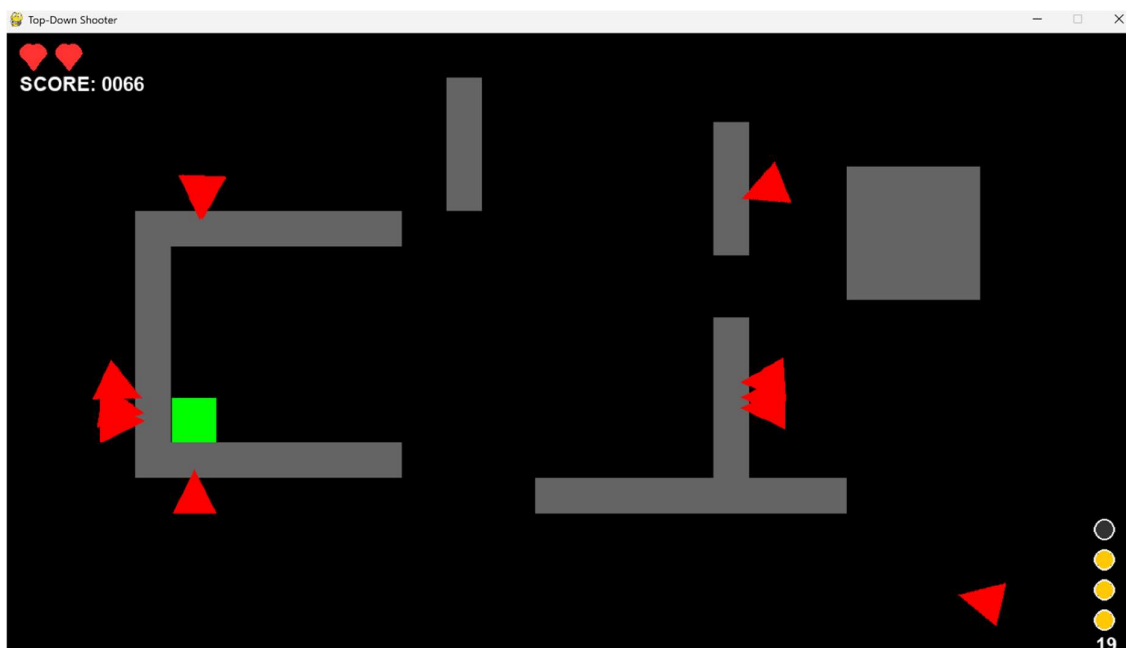


Abb. 13 TD-Shoot Abnahmetests 9/11



Abb. 14 TD-Shoot Abnahmetests 10/11

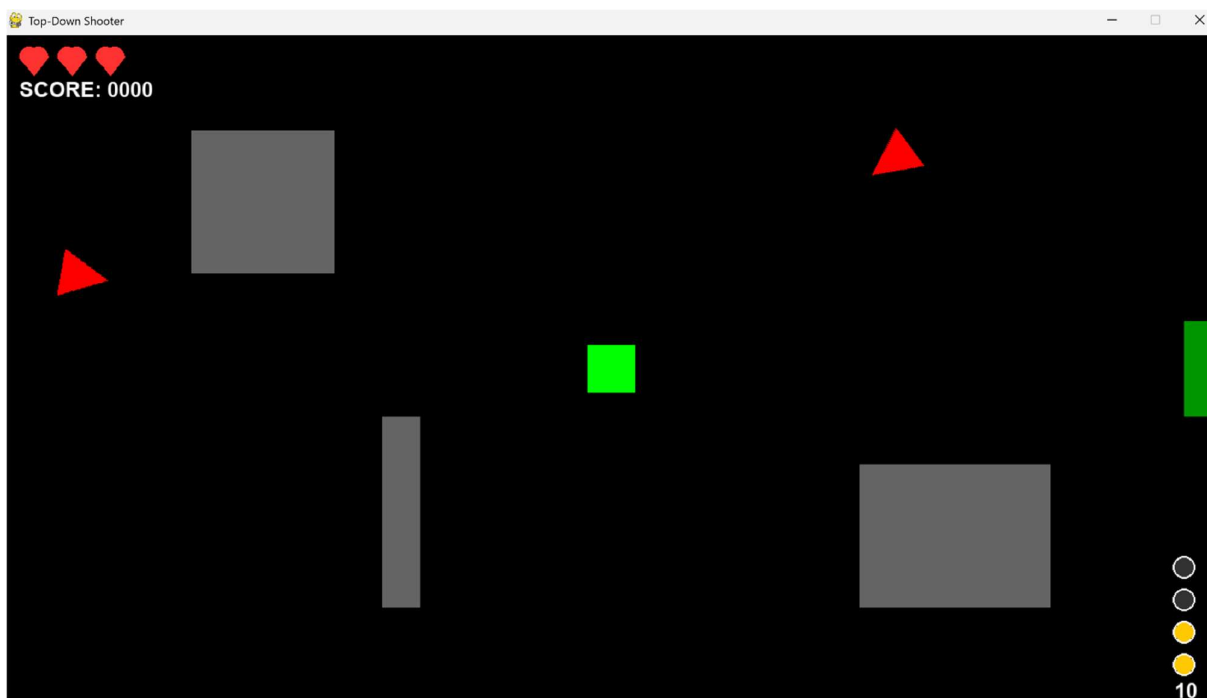


Abb. 15 TD-Shoot Abnahmetests 11/11

14 Schlussbericht

Detaillierter Schlussbericht für TD-Shoot erstellt. Analysiert wurden:

Alle Projektdaten:

- Zielerreichung (4/4 erfüllt)
- Meilensteine (M1-M4 termingerecht)
- Tests (100% Erfolgsrate)
- Risiken (R4 eingetreten, gut gemanagt)
- Aufwände (+12% Mehraufwand)
- Team-Reflexionen
- Stärken/Schwächen und konkrete Empfehlungen

Umfassende Bewertung des TD-Shoot-Projekts als Erfolg trotz QA-Ausfall.

14.1 Management Summary

Das TD-Shoot Projekt wurde im Rahmen des Moduls 306A als HERMES-5-konformes IT-Kleinprojekt erfolgreich durchgeführt. Innerhalb von sechs Kurstagen entwickelte das vierköpfige Team einen funktionsfähigen Top-Down-Shooter als Python-EXE-Anwendung mit zwei spielbaren Levels, vollständiger Testdokumentation und strukturierter Projektorganisation via GitHub.

Kernergebnisse:

- Lauffähiges Spielprodukt mit allen definierten Core-Features
- 100% Unit-Test-Erfolgsrate (13/13 Tests bestanden)
- 100% Abnahmetest-Erfolgsrate (6/6 Testfälle Fehlerklasse 0)
- Vollständige HERMES-Dokumentation (PIA, PMP, GDD, Testkonzepte, Schlussbericht)
- Fristgerechte Erreichung aller vier Meilensteine (M1-M4)

Das Projekt demonstriert erfolgreiche Zusammenarbeit unter Zeitdruck trotz personeller Herausforderungen in der Endphase.

14.2 Zielerreichung

14.2.1 Ziel Z1: Spielbares MVP innerhalb von sechs Kurstagen

Status: Erreicht

Das definierte Minimal Viable Product wurde vollständig umgesetzt und übertroffen:

Geplante Features (erfüllt):

- Ein spielbares Level mit Start/Ende-Bedingungen
- Präzise Tastatur- und Maussteuerung (WASD + Mausklick)
- Gegnertyp mit Bewegungs-KI und Schadenslogik
- Punktesystem mit visueller Anzeige
- Stabile EXE-Anwendung ohne Abstürze

Zusätzlich realisiert:

- Zweites Level mit erhöhter Komplexität
- Progressive Schwierigkeitssteigerung (Gegnergeschwindigkeit, Spawn-Rate)
- Munitionssystem mit Reload-Mechanik und Progressionsbelohnung (+5 Magazinkapazität alle 100 Punkte)
- Pause-Funktionalität (ESC-Taste)
- Game-Over-Screen mit korrekter State-Reset-Logik

Bewertung: Das MVP-Ziel wurde nicht nur erreicht, sondern durch sinnvolle Zusatzfeatures erweitert, ohne den Zeitrahmen zu sprengen.

14.2.2 Ziel Z2: Strukturierte Projektplanung nach HERMES 5

Status: Erreicht

Alle HERMES-Phasen wurden methodenkonform durchlaufen:

Phase Initialisierung (M1 - 03.03.2026):

- Projektinitialisierungsantrag (PIA) erstellt
- Stakeholder- und Risikoanalyse dokumentiert
- GitHub-Repository mit Ordnerstruktur (/docs, /src) eingerichtet
- Rollen klar definiert (PL, BA, DEV, QA)

Phase Konzept (M2 - 10.03.2026):

- Projektmanagementplan (PMP) mit Zeitplanung
- Game Design Document (GDD) mit vollständiger Architektur
- Teststrategie mit Unit- und Abnahmetests definiert
- funktionale und nicht-funktionale Anforderungen spezifiziert

Phase Realisierung (M3 - 17./18.03.2026):

- Strukturierte Implementierung nach EVA-Prinzip
- Kontinuierliche GitHub-Nutzung (Issues, Branches, Pull-Requests)
- Unit-Tests parallel zur Entwicklung
- Statusberichte mit transparenter Fortschrittskommunikation

Phase Einführung (M4 - 24./25.03.2026):

- EXE-Build mit Startanleitung bereitgestellt
- Schlusssdokumentation und Lessons Learned
- PowerPoint-Präsentation für Stakeholder-Abschlusspräsentation (31.03.2026)

Bewertung: Die HERMES-Methodik wurde konsequent angewendet.
Alle geforderten Dokumente liegen vor gemäss entsprechenden Anforderungen.

14.2.3 Ziel Z3: Aufbau von Praxiswissen in Game-Development und Python

Status: Erreicht

Das Team sammelte umfassende Praxiserfahrung in neuen Technologiebereichen:

Game-Development-Kompetenzen:

- Implementierung eines Game-Loops mit State-Machine-Muster
- Sprite-basierte Architektur mit Pygame
- Kollisionserkennung und Physik-Berechnungen (Vektorrechnung, Normalisierung)
- Pathfinding-KI für Gegnerverfolgung
- UI/HUD-Design und Rendering-Logik

Python-EXE-Build:

- Erstellung lauffähiger executables aus Python-Code
- Dependency-Management mit requirements.in
- Build-Prozess-Dokumentation im README

Technische Architektur:

- EVA-Prinzip (Eingabe-Verarbeitung-Ausgabe)
- Klassenbasierte Objektstruktur (Player, Enemy, Projectile)
- Level-Manager und Config-Management
- Unit-Testing mit pytest-Framework

Bewertung: Dokumentierte Reflexionen im Arbeitsjournal zeigen klaren Wissenszuwachs. DEV-Reflexion betont erfolgreiche Strukturierung und sofortiges Testing. BA-Reflexion dokumentiert Git-Workflow-Lernkurve.

14.2.4 Ziel Z4: Hohe Nachvollziehbarkeit durch GitHub-Nutzung

Status: Erreicht

GitHub wurde als zentrale Kollaborations- und Qualitätsplattform konsequent genutzt:

Versionierung:

- Vollständiger Source-Code im /src-Ordner
- Alle Dokumente im /docs-Ordner versioniert
- Dokumentversionen mit klarer Änderungskontrolle (1.0 bis 2.0)

Zusammenarbeit:

- Issues für Task-Tracking und Fehlermanagement
- Branch-Strategie für parallele Entwicklung
- Pull-Requests mit Code-Reviews vor Merge
- README mit Projektübersicht und Startanleitung

Qualitätssicherung:

- Pytest-Tests im Repository dokumentiert
- Testprotokolle als Anhänge verfügbar
- Bug-Tracking (Bug 1: Gegner-Glitch, Bug 2: Score-Reset-Problem)

Bewertung: Das Repository erfüllt alle geforderten Vorgaben (A10). Transparente Nachvollziehbarkeit aller Projektaktivitäten ist gegeben.

14.3 Anforderungserfüllung

Alle 10 definierten Anforderungen wurden vollständig erfüllt:

ID	Anforderung	Status	Nachweis
A1	Mindestens ein vollständiges Level mit Start/Ende-Bedingung	Erfüllt	Zwei Level implementiert, Testfall F5 bestanden
A2	Präzise Tastatur-/Maussteuerung (WASD, Schiessen)	Erfüllt	Steuerung funktional, Testfall F2 bestanden
A3	Gegnertyp mit Bewegung und Schadenslogik	Erfüllt	KI-Pathfinding implementiert, Testfall F3 bestanden
A4	Punktestand-Anzeige mit korrektem Reset	Erfüllt	Score-System funktional, Testfall F4 bestanden
A5	Stabile Ausführung ohne Abstürze	Erfüllt	Alle Tests Fehlerklasse 0, kein Crash dokumentiert
A6	Python-EXE mit Startanleitung	Erfüllt	TD-Shooter.exe + README.md vorhanden
A7	GDD mit Anforderungen dokumentiert	Erfüllt	GDD in Anhang 6, detaillierte Spielbeschreibung
A8	Testfälle und Testprotokolle	Erfüllt	13 Unit-Tests + 6 Abnahmetests dokumentiert
A9	HERMES-Dokumente (PIA, PMP, GDD, Berichte)	Erfüllt	Alle Dokumente in /docs vorhanden
A10	GitHub-Vorgaben (README, Issues, Branches, Reviews)	Erfüllt	Repository erfüllt alle Kriterien

Erfüllungsrate: 10/10 (100%)

14.4 Unit-Tests / Gameplay-Tests

Mit pytest wurden 13 automatisierte Tests durchgeführt, die die Kernlogik isoliert verifizieren:

Player-Tests (6/6 bestanden):

- Korrekte HP-Reduktion bei Treffern
- Sterbe-Status bei 0 HP
- Munitionsverwaltung (Abzug pro Schuss, Reload-Status)

Enemy-Tests (4/4 bestanden):

- Vektorberechnung für Spielerverfolgung
- Schadensannahme bei Kollision
- Score-Wert-Rückgabe beim Ableben

Projectile-Tests (3/3 bestanden):

- Richtungsvektor-Normalisierung
- Konstante Geschwindigkeit pro Frame
- Kill-Funktionalität

Unit-Test-Erfolgsrate: 13/13 (100%)

14.5 Abnahmetests

Sechs manuelle Abnahmetests gegen die funktionalen Anforderungen:

ID	Beschreibung	Fehlerklasse	Status
F1	Startup (Spiel startet korrekt)	0 (fehlerfrei)	Erfüllt
F2	Schussfunktion (Projektile treffen Gegner)	0 (fehlerfrei)	Erfüllt
F3	Health-System (Leben wird korrekt abgezogen)	0 (fehlerfrei)	Erfüllt
F4	Score-System (Punkte werden gezählt)	0 (fehlerfrei)	Erfüllt
F5	Levelsystem (Level 2 wird geladen)	0 (fehlerfrei)	Erfüllt
F6	Spielende (Game-Over und Reset funktionieren)	0 (fehlerfrei)	Erfüllt

Abnahmetest-Erfolgsrate: 6/6 (100%)

15 Reflexionen des Projektteams

15.1 Projektleitung (Stefanie Gerber)

Höhepunkte:

- Erfolgreiche Koordination eines komplexen Projekts mit knappen Zeitressourcen
- Proaktive Krisenintervention bei QA-Ausfall und technischen Blockaden
- HERMES-Dokumentation wurde strukturiert und fristgerecht geliefert

Herausforderungen:

- Kommunikation war teilweise zu indirekt, direktere Ansprache hätte Zeit gespart
- Einzelne Teammitglieder zeigten niedrige Initiative

Learnings:

- "Klarer, direkter kommunizieren" (Reflexion Tag 1)
- "Wissensmissstände führen oft zu Komplikationen, die in Mehraufwand enden" (Reflexion Tag 2)
- Balance zwischen Mikromanagement und Delegation ist kritisch

Empfehlung für zukünftige Projekte:

- Wöchentliche 1:1-Gespräche mit Rollenmitgliedern für frühzeitige Problemerkennung
- Klare Eskalationswege bei technischen Blockaden definieren

15.2 Development (Christophe Grädel)

Höhepunkte:

- "Programmierung verlief planmässig und strukturiert" (Reflexion Tag 3)
- Sofortiges Testing ermöglichte frühes Fehlerfindung
- Technische Architektur (EVA-Prinzip, Sprite-System) war robust

Herausforderungen:

- Git-Workflows für Teammitglieder erklären war zeitaufwändig
- Build-Prozess für EXE war dokumentationsarm, Learning-by-Doing

Learnings:

- Klare Code-Struktur reduziert Debugging-Zeit massiv
- Pair-Programming bei kritischen Features wäre effizienter gewesen

Empfehlung für zukünftige Projekte:

- Technische Onboarding-Session am Projekttag 1 für alle Tools (Git, pytest, Pygame)
- Continuous Integration für automatisiertes Testing einrichten

15.3 Business Analyse (Benjamin Phengrasamy)

Höhepunkte:

- Terminplan wurde erfolgreich umgesetzt und mit Team abgestimmt
- PowerPoint-Präsentation für Stakeholder-Abschluss finalisiert
- Arbeitszeiten Dokumentation verlief gut

Herausforderungen:

- "Pushen über GitBash-Terminal in GitHub war meine Herausforderung" (Reflexion Tag 4)
- Abhängigkeit von Teamkollegen für Git-Support

Learnings:

- Mit Hilfe von Team und Kollegen Git-Workflow erlernt
- Proaktives Nachfragen beschleunigt Lernkurve

Empfehlung für zukünftige Projekte:

- Frühzeitiges Hands-on-Training für Git (nicht nur theoretisch)
- Checklisten für wiederkehrende Tasks (z.B. Commit-Push-Workflow)

15.4 Quality Assurance (Sujanthan Suntheralingam)

Anmerkung: QA verliess das Team am 24.03.2026 aus persönlichen Gründen. Testkonzepte und initiale Testfälle wurden am 10.03 von DEV übernommen und durchgeführt. Nach Ausfall übernahmen PL, BA und DEV die QA-Aufgaben.

15.5 Gesamtfazit und Empfehlungen

Projekterfolg: Ja

Das TD-Shoot-Projekt ist nach allen definierten Kriterien ein Erfolg:

1. Produktqualität: Lauffähiges Spiel mit allen Core-Features und Zusatzfunktionen
2. HERMES-Konformität: Alle Dokumente und Meilensteine fristgerecht geliefert
3. Testqualität: 100% Unit- und Abnahmetest-Erfolgsrate
4. Teamleistung: Konstruktive Zusammenarbeit trotz personeller Herausforderung
5. Lernziele: Praxiswissen in Game-Development, Python-EXE und HERMES erworben

15.5.1 Kritische Erfolgsfaktoren

Was das Projekt erfolgreich machte:

- Klare MVP-Definition verhinderte Scope-Verlust
- Iteratives Vorgehen (frühes Testing, Prototyping) reduzierte Risiken
- Transparente Kommunikation über GitHub und Meetings
- Flexibilität bei QA-Ausfall durch Wissensverteilung
- Stakeholder-Einbindung sicherte Alignment bei Meilensteinen

15.5.2 Verbesserungspotenziale

Teamebene:

- Technisches Onboarding: Erste Halbtage für Git/Tools-Training
- Backup-Rollen: Jede kritische Aufgabe mit Stellvertreter:in
- Daily Stand-ups: 10-Minuten-Check-ins statt wöchentlicher Meetings
- Pair-Programming: Bei komplexen Features für Wissenstransfer

Prozessebene:

- Continuous Integration: Automatisierte Tests bei jedem Commit
- Code-Review-Kultur: Mehr Zeit für qualitative Pull-Request-Reviews
- Risikomonitoring: Wöchentliche Risiko-Updates statt nur bei Projektstart
- Lessons-Learned-Sessions: Nach jedem Meilenstein, nicht nur am Ende

Technische Ebene:

- Dokumentation: README sollte schrittweise Build-Anleitung enthalten

15.5.3 Ausblick: Potenzielle Weiterentwicklung

Obwohl das Projekt abgeschlossen ist, gäbe es Potenzial für Folgeprojekte:

Feature-Backlog (nicht umgesetzt):

- Mehrere Ggnerarten mit unterschiedlichen Verhaltensweisen
- Power-Ups und Waffen-Upgrades
- Online-Highscore-System
- Story-Modus
- Multiplayer-Funktionalität
- Sound-Engine mit dynamischer Musik

16 Schlusswort

Das TD-Shoot-Projekt hat gezeigt, dass ein kleines, fokussiertes Team unter Zeitdruck ein qualitativ hochwertiges IT-Produkt liefern kann, wenn methodische Disziplin (HERMES 5), technische Kompetenz und konstruktive Zusammenarbeit zusammenkommen. Die grösste Herausforderung, der Ausfall des QA Teammitglieds in der kritischen Endphase wurde durch Flexibilität und gegenseitige Unterstützung gemeistert. Die 100%ige Testabdeckung trotz dieser Erschwernis unterstreicht die Professionalität des Teams. Alle vier Projektziele wurden erreicht oder übertroffen. Das lauffähige Spiel, die vollständige HERMES-Dokumentation und die strukturierte GitHub-Nutzung bilden eine solide Grundlage für die Abschlusspräsentation am 31.03.2026

Dank gilt allen Beteiligten:

- Dem Auftraggeber Michal Durik für klare Rahmenbedingungen und konstruktives Feedback
- Dem Projektteam für Durchhaltevermögen und Lernbereitschaft
- Der Schule für Bereitstellung der Kommunikationsplattform und methodischen Rahmens

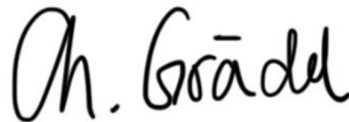
Das Projekt endet mit einem funktionsfähigen Produkt, wertvollen Lernerfahrungen und einem gestärkten Teamverständnis für professionelle IT-Projektarbeit.

Projektleitung
Stefanie Gerber



25.03.2026

Development
Christophe Grädel



Business Analyse
Benjamin Phengrasamy



16 Abbildungsverzeichnis

Abb. 1 TD-Shoot UI Konzept	21
Abb. 2 TD-Shoot Unit/Gameplay Tests 1/3	22
Abb. 3 TD-Shoot Unit/Gameplay Tests 2/3	22
Abb. 4 TD-Shoot Unit/Gameplay Tests 3/3	22
Abb. 5 TD-Shoot Abnahmetests 1/11	25
Abb. 6 TD-Shoot Abnahmetests 2/11	25
Abb. 7 TD-Shoot Abnahmetests 3/11	26
Abb. 8 TD-Shoot Abnahmetests 4/11	27
Abb. 9 TD-Shoot Abnahmetests 5/11	28
Abb. 10 TD-Shoot Abnahmetests 6/11	28
Abb. 11 TD-Shoot Abnahmetests 7/11	29
Abb. 12 TD-Shoot Abnahmetests 8/11	29
Abb. 13 TD-Shoot Abnahmetests 9/11	30
Abb. 14 TD-Shoot Abnahmetests 10/11	31
Abb. 15 TD-Shoot Abnahmetests 11/11	31

17 Anhänge

Anhangsverzeichnis

1	Anhang Projektinitialisierungsantrag	44
2	Anhang Arbeitszeiten.xlsx	45
3	Anhang PMP.md	46
4	Anhang Risikoliste.md	47
5	Anhang Stakeholderliste.md	48
6	Anhang GDD.md (Game Design Dokument)	49
7	Anhang requirements.in	50
8	Anhänge Source	51
9	Anhang Unit/Gameplay Tests	55
10	Anhang README.md	56

1 Anhang Projektinitialisierungsantrag

[illegible][illegible]

Klausur	
Prüfungsausschuss	
Inhaltsverzeichnis	
1. Ausgangslage	
2. Ziel	
3. Rahmenbedingungen	
4. Aufwand	
5. Risiko	
6. Nutzen	
7. Ressourcen	
8. Kommunikation	
9. Risiken	
10. Projektabschlussempfehlung	

Abschlussverzeichnis

	Problemlösungsschritte
9 Risiken	10 Risiken
• Welche Risiken sind für die Phase Initiierung relevant? • Woher sehen wir die Extrinsische Wirtschaftlichkeit und das Mass der Auswirkungen des Risikos an? • Wozum wir kognitiven Massnahmen bedürfen, um bestimmte Risiken zu minimieren?	
R1 - Ziel nicht für Initiierung nicht aus	
• Identifikation: Initiierungsvorgängen (Zeit, Scope, Prio, Risiko, Ressourcen) gegenüber das verfügbare Zeitfenster im Hinblick	
• Wirtschaftlichkeit: nicht	
• Auswirkung: hoch	
• Massnahmen: Mass Agieren, Transparenz zu Akteuren, offene Punkte als Topic für weitere massnahmen	
R2 - Unklare Ziele / Scope des Projekts	
• Beschreibung: Team und am anhebel der Initiierung nicht auf ein realistisches MPF für den Team/Steuerer setzen	
• Wirtschaftlichkeit: nicht	
• Auswirkung: hoch	
• Massnahmen: MPF sukzessive hochziehen (1. Level Basis-Funktion, Prio, anschließend bei Unklarheiten weitere Funktionen einbeziehen oder Change Request)	
R3 - Technische Risiken und Python / Excel / Outlook	
• Beschreibung: Probleme mit Python-Umgebung, Excel/Outlook gegen Github/Workflow verknüpfen ist die korrekte Installation notwendig	
• Wirtschaftlichkeit: nicht	
• Auswirkung: mittel	
• Massnahmen: Rich Text/Word und complex/HTML/PHP/Excel anlegen, minimieren Bald Mail-Prozess besser gestalten, technische Support schnell anbieten und unterstützen	

Scrum/Agile 2023 SS 24

Seite 10

[illegible]

10 Projektfollowingenauftrag

Herrni erteilt der Auftraggeber den Auftrag zur Durchföhrung der Initiellierungsephase des Projektes:

Der Auftraggeber (für die Phase Initiellierung):
(OM, Datum, Unterschrift)

Der Projektleiter (für die Phase Initiellierung):
(OM, Datum, Unterschrift)

3 Anhang PMP.md

Projektmanagementplan (PMP) – TD-Shooter

Dieser Plan beschreibt die Steuerung und Umsetzung des Projekts TD-Shooter im Rahmen des Moduls 306A, basierend auf der Methode HERMES 5 (Szenario: Kleinprojekt).

1. Phasen, Zeitplan und Meilensteine

Das Projekt ist auf eine Laufzeit von ca. 6 Tagen ausgelegt und folgt dem Phasenmodell für Kleinprojekte.

Phase	Zeitraum	Fokus & Aktivitäten	Meilenstein (Entscheid)
Initialisierung	03.03.2026	Projektstart, Repository-Setup, PIA, Stakeholder & Risiken	M1: Projektfreigabe
Konzept	Tag 2	Erstellung GDD, PMP, Teststrategie und Architekturplan	M2: Konzeptfreigabe
Realisierung	Tag 3 – 4	Iterative Entwicklung, Feature-Implementierung, Testprotokolle	M3: Lauffähiger Prototyp
Einführung	Tag 5 – 6	Abnahmetests, EXE-Release, Abschlussbericht & Präsentation	M4: Schlussabnahme

2. Projektorganisation und Rollen

Das Team verteilt die Rollen gemäss HERMES-Standard, angepasst auf die Teamgrösse von vier Personen.

Rolle	Person	Hauptaufgaben
Projektleitung (PL)	Stefanie Gerber	Termine, Scope, Risiko-Management, Stakeholder-Kommunikation.
Business Analyst (BA)	Christophe Grädel	Anforderungen, GDD-Pflege, Definition der Abnahmekriterien.
Entwicklung (DEV)	Benjamin Phengrasamy	Software-Architektur, Implementation (Python/Pygame), Build-Prozess.
Qualitätssicherung (QA)	Sujanthan Suntheralingam	Testkonzept, Testprotokolle, Durchführung der Abnahmetests.

Stellvertretung: Jede Kernaufgabe wird durch eine zweite Person mitverfolgt. Die laufende Dokumentation im GitHub-Repository stellt sicher, dass das Projekt auch bei Ausfällen fortgeführt werden kann.

3. Planung und Steuerung

Die Steuerung erfolgt aktiv über das Repository und tägliche Abstimmungen.

- **Daily Sync:** Ein kurzes 5–10 minütiges Meeting zu Beginn jedes Kurstages (Fortschritt, Blocker, Tagesziele).
- **Task-Management:** Alle Aufgaben werden als GitHub-Issues erfasst (inkl. Zuweisung und Zieltermin).
- **Versionskontrolle:** Nutzung von **Feature-Branches**. Änderungen werden ausschliesslich via Pull-Requests nach einem Review in den `main`-Zweig gemergt.
- **Berichterstattung:** Die PL erstellt während der Realisierung Statusberichte und informiert den Auftraggeber bei Meilensteinen.

4. Qualitätssicherung

Um eine gleichbleibende Qualität sicherzustellen, gilt die folgende **Definition of Done (DoD)** für jeden Arbeitsauftrag:

- Code ist im Repository eingchecked und fehlerfrei ausführbar.
- Die Funktionalität wurde gemäss Anforderungen getestet.
- Dokumentation (GDD, Testprotokoll) wurde bei Bedarf aktualisiert.
- Mindestens ein Peer-Review durch ein anderes Teammitglied wurde durchgeführt.

Die HERMES-Dokumente (PIA, PMP, Testkonzept etc.) werden im Verzeichnis `/docs` gepflegt und gegengelesen.

5. Kommunikation

Kanal	Empfänger	Zweck
Präsenz / Daily	Team	Koordination der täglichen Aufgaben und Probleme.
MS Teams / Discord	Team	Kommunikation im Homeoffice oder bei dringenden Fragen.
GitHub Issues/PRs	Team	Dokumentation technischer Entscheidungen und Code-Reviews.
Präsentationen	Auftraggeber	Abnahme der Meilensteine M1 bis M4.

6. Risiko- und Change-Management

Risiko-Management

Risiken werden nach Eintrittswahrscheinlichkeit (W) und Schadensausmass (S) auf einer Skala von 1 (niedrig) bis 3 (hoch) bewertet.

Risiko	W	S	Massnahme
Zeitknappheit	3	3	Konzentration auf MVP (Minimum Viable Product); "Nice-to-haves" streichen.
Technische Hürden	2	2	Frühzeitige Spikes/Prototypen für komplexe Mechaniken.
Ausfall Teammitglied	1	3	Strikte Dokumentation und Code-Reviews zur Wissensverteilung.

Change-Management

Anpassungen am Scope oder den Anforderungen werden als **Change-Requests** behandelt:

1. Erfassung als GitHub-Issue mit Label `change`.
2. Diskussion im Team über Auswirkungen auf Zeit und Ressourcen.
3. Entscheid durch die Projektleitung in Abstimmung mit dem Auftraggeber.

4 Anhang Risikoliste.md

Risikoliste TD-Shooter

Stand: 03.03.2026

Legende:

- W = Wahrscheinlichkeit (niedrig / mittel / hoch)
- A = Auswirkung (niedrig / mittel / hoch)

R1 – Zeit reicht für Initialisierung/Konzept nicht aus

- Beschreibung: Umfang der Aufgaben (PIA, Stakeholderliste, Risikoliste, GDD, PMP) sprengt das verfügbare Zeitfenster.
- W: mittel
- A: hoch
- Massnahmen:
 - Klare Agenda und Timeboxing pro Teilaufgabe.
 - Offene Punkte als To-dos für später festhalten.
 - Fokus zuerst auf Pflichtdokumente für Meilenstein.

R2 – Unklarer Scope / MVP des Spiels

- Beschreibung: Team einigt sich nicht auf ein realistisches MVP; es werden zu viele Features geplant.
- W: mittel
- A: hoch
- Massnahmen:
 - MVP im GDD schriftlich festhalten (1 Level, Basis-Features).
 - PL entscheidet bei Unklarheiten.
 - Erweiterungen nur über dokumentierte Change Requests.

R3 – Technische Probleme mit Python / EXE / GitHub

- Beschreibung: Probleme mit Entwicklungsumgebung, Libraries, EXE-Build oder GitHub-Workflow verzögern die Umsetzung.
- W: mittel
- A: mittel
- Massnahmen:
 - Früh ein Minimalprojekt („Hello World“-Spiel) erstellen und bis EXE/Run testen.
 - GitHub-Zugriff und Branching vorab prüfen.
 - Technische Issues sofort erfassen, priorisieren und einem DEV zuweisen.

R4 – Ausfall von Teammitgliedern

- Beschreibung: Krankheit oder Ausfall eines Teammitglieds führt zu Engpässen bei Code oder Dokus.
- W: niedrig
- A: hoch
- Massnahmen:
 - Aufgaben und Wissen verteilen (kein Single Point of Failure).
 - GitHub konsequent nutzen (Code, Dokus, Protokolle aktuell halten).
 - Backup-Verantwortliche pro Rolle definieren (z.B. 2. DEV, 2. QA).

R5 – ungenügende Testabdeckung

- Beschreibung: Zu wenig Zeit für Tests, Fehler fallen erst bei Abnahme auf.
- W: mittel
- A: mittel
- Massnahmen:
 - Einfaches Testkonzept früh erstellen.
 - Smoke-Tests nach jedem grösseren Merge.
 - QA-Check vor Meilensteinen einplanen.

5 Anhang Stakeholderliste.md

Stakeholderliste TD-Shooter

Stand: 03.03.2026

1. Auftraggeber

- Name/Rolle: Michal Durik, Dozent
- Interesse: Bewertung von Produkt und HERMES-Dokumentation, Einhaltung Modulvorgaben
- Einfluss: Hoch
- Informationsbedarf: Status je Phase, Meilenstein-Ergebnisse (PIA, GDD, PMP, Tests, Abschluss)

2. Projektteam

- Namen: Stefanie Gerber, Christophe Grädel, Benjamin Phengrasamy, Sujanthan Suntheralingam
- Rollen: PL, BA, DEV, QA (teilweise kombiniert)
- Interesse: Erfolgreiches Projekt, gute Note, lernbarer Code und saubere Dokus
- Einfluss: Hoch
- Informationsbedarf: Aufgaben, Termine, Risiken, Änderungsentscheide

3. Endbenutzer

- Name/Rolle: Mitschüler / typische Spieler des TD-Shooters
- Interesse: Spielspass, einfache Bedienung, stabile Version
- Einfluss: Mittel
- Informationsbedarf: Startanleitung, Steuerung, bekannte Einschränkungen

4. Schule / IT-Umgebung

- Name/Rolle: Schul-IT / Schulorganisation
- Interesse: Spiel läuft auf Schulrechnern ohne Probleme, keine Sicherheitsrisiken
- Einfluss: Mittel
- Informationsbedarf: Technische Anforderungen (Python-Version, EXE, benötigte Rechte)

5. GitHub-Plattform

- Name/Rolle: GitHub als Dienst
- Interesse: Korrekte Nutzung gemäss Nutzungsbedingungen
- Einfluss: Niedrig (aber technisch kritisch)
- Informationsbedarf: Keine direkte Kommunikation

8 Anhänge Source

```
src > _init_.py
1 # from .file import Function/ Classe
2 from .player import Player
3 from .projectile import Projectile
4 from .enemy import Enemy
5 from .level_manager import LevelOne, LevelTwo
6 from .ui_manager import UIManager
7 from . import config
```

```
src > config.py > ...
1 # main
2 WIDTH = 1280
3 HEIGHT = 700
4
5 COLOR1 = (255, 255, 255)
6 COLOR2 = (255, 255, 255)
7 COLOR3 = (0, 0, 0)
8 RED = (255, 0, 0)
9 GRAY = (100, 100, 100)
10
11 # PLAYER
12 PLAYER_HEALTH = 3
13 PLAYER_SPEED = 150
14 PLAYER_START_AMMO = 10
15 PLAYER_MAX_AMMO = 20
16 RELOAD_DURATION = 2000
17 PLAYER_AMMO_FACTOR = 5
18
19 # PROJECTILE
20 BULLET_SPEED = 600
21
22 # ENEMY
23 ENEMY_HEALTH = 1
24 ENEMY_SPEED = 110
25 ENEMY_SPEED_FACTOR = 15
26 ENEMY_ATTACK_DMG = 1
27 ENEMY_SCORE_VALUE = 1
28
29 SPWAN_TIMER = 0
30 SPWAN_COOLDOWN = 1500
31 ENEMY_SPAWN_SCORE_STEP = 50
32 ENEMY_SPAWN_INCREMENT = 1
34 # UI MANAGER
35 COLOR_HP = (255, 50, 50)
36 COLOR_AMMO = (255, 200, 0)
37 COLOR_RELOAD = (100, 100, 100)
38 COLOR_TEXT = (255, 255, 255)
```

```
src > enemy.py > Enemy > look_at
1 from . import config
2 import pygame
3 import math
4
5 class Enemy(pygame.sprite.Sprite):
6     def __init__(self, x, y, speed):
7         super().__init__()
8         self.original_image = pygame.Surface((50, 50), pygame.SRCALPHA)
9         enemy_color = config.RED
10         points = [(25, 0), (0, 50), (50, 50)]
11         pygame.draw.polygon(self.original_image, enemy_color, points)
12
13         self.image = self.original_image.copy()
14         self.rect = self.image.get_rect()
15         self.rect.center = (x, y)
16
17         self.hitbox = pygame.Rect(0, 0, 30, 30)
18         self.hitbox.center = (x, y)
19
20         self.hp = config.ENEMY_HEALTH
21         self.speed = speed
22         self.speed_factor = config.ENEMY_SPEED_FACTOR
23         self.pos = pygame.Vector2(x, y)
24
25     def update(self, player_pos, dt, obstacles, *args):
26         direction = pygame.Vector2(player_pos) - self.pos
27
28         if direction.length() > 0:
29             velocity = direction.normalize() * self.speed * dt
30
31             # --- X-ACHSE ---
32             self.pos.x += velocity.x
33             self.hitbox.centerx = round(self.pos.x)
34
35             for wall in obstacles:
36                 if self.hitbox.colliderect(wall.rect):
37                     if velocity.x > 0:
38                         self.hitbox.right = wall.rect.left
39                     elif velocity.x < 0:
40                         self.hitbox.left = wall.rect.right
41                     self.pos.x = float(self.hitbox.centerx)
42
43             # --- Y-ACHSE ---
44             self.pos.y += velocity.y
45             self.hitbox.centery = round(self.pos.y)
46
47             for wall in obstacles:
48                 if self.hitbox.colliderect(wall.rect):
49                     if velocity.y > 0:
50                         self.hitbox.bottom = wall.rect.top
51                     elif velocity.y < 0:
52                         self.hitbox.top = wall.rect.bottom
53                     self.pos.y = float(self.hitbox.centery)
54
55             # --- GRAFIC & ROTATION ---
56             angle = direction.angle_to(pygame.Vector2(0, -1))
57             self.image = pygame.transform.rotate(self.original_image, angle)
58             self.rect = self.image.get_rect(center=self.hitbox.center)
59
60     def look_at(self, target_pos):
61         directions = pygame.Vector2(target_pos) - self.rect.center
62         angle = directions.angle_to(pygame.Vector2(0, -1))
63         self.image = pygame.transform.rotate(self.original_image, angle)
64
65     def take_hit(self, damage):
66         self.hp -= damage
67         if self.hp <= 0:
68             self.kill()
69         return config.ENEMY_SCORE_VALUE
```

```

src > level_manager.py > _
1 import pygame
2 from . import config
3
4 class Obstacle(pygame.sprite.Sprite):
5     def __init__(self, x, y, width, height):
6         super().__init__()
7         self.image = pygame.Surface((width, height))
8         self.image.fill(config.GRAV)
9         self.rect = self.image.get_rect(topleft=(x, y))
10
11     def update(self, *args):
12         pass
13
14 class LevelExit(pygame.sprite.Sprite):
15     def __init__(self, x, y, width, height):
16         super().__init__()
17
18     self.image = pygame.Surface((width, height), pygame.SRCALPHA)
19     self.image.fill((0, 255, 0, 150))
20     self.rect = self.image.get_rect(topleft=(x, y))
21
22     def update(self, *args):
23         pass
24
25 class LevelOne:
26     def __init__(self):
27         super().__init__()
28
29         self.obstacles = pygame.sprite.Group()
30
31         level_data = [
32             (200, 100, 150, 150),
33             (400, 400, 40, 200),
34             (200, 100, 100, 100),
35             (900, 450, 200, 150),
36         ]
37
38         for obs in level_data:
39             wall = Obstacle(*obs)
40             self.obstacles.add(wall)
41
42         self.exit_portal = LevelExit(config.WIDTH - 40, config.HEIGHT // 2 - 50, 40, 100)
43
44 class LevelTwo:
45     def __init__(self):
46         super().__init__()
47
48         self.obstacles = pygame.sprite.Group()
49         level_data = [
50             # 1-Form
51             (800, 100, 40, 150),
52             (800, 320, 40, 180),
53             (600, 500, 350, 40),
54             (950, 150, 150, 150),
55             (500, 50, 40, 150),
56             # dead end
57             (150, 200, 300, 40),
58             (150, 200, 40, 300),
59             (150, 460, 300, 40),
60         ]
61
62         for obs in level_data:
63             self.obstacles.add(Obstacle(*obs))
64
65         self.exit_portal = None

```

```

src > main.py > %Game > _init_
1 import pygame
2 import random
3 from . import player, projectile, enemy, level_manager, ui_manager, config
4
5 class Game:
6     def __init__(self):
7         pygame.init()
8         self.ui_manager = ui_manager.UIManager()
9         self.screen = pygame.display.set_mode((config.WIDTH, config.HEIGHT))
10        pygame.display.set_caption("Top-Down Shooter")
11        self.clock = pygame.time.Clock()
12        self.running = True
13
14        self.notification_text = ""
15        self.notification_timer = 0
16
17        self.font = pygame.font.SysFont("Arial", 48, bold=True)
18        self.small_font = pygame.font.SysFont("Arial", 22, bold=True)
19
20        self.state = "MENU"
21
22        def new(self):
23            """Initialisiert eine komplett neue Spielrunde"""
24            self.current_enemy_speed = config.ENEMY_SPEED
25            self.current_spawn_cooldown = config.SPAWN_COOLDOWN
26            self.notification_text = ""
27            self.notification_timer = 0
28
29            self.all_sprites = pygame.sprite.Group()
30            self.bullets = pygame.sprite.Group()
31            self.enemies = pygame.sprite.Group()
32
33            self.player = player.Player()
34            self.all_sprites.add(self.player)
35
36            self.current_level_num = 1
37            self.load_level(level_manager.LevelOne())
38
39            self.score = 0
40            self.start_time = pygame.time.get_ticks()
41            self.last_spawn_time = 0
42
43        def load_level(self, level_obj):
44            self.current_level_obj = level_obj
45            self.obstacles = self.current_level_obj.obstacles
46            self.level_exit = getattr(level_obj, 'exit_portal', None)
47
48            self.all_sprites.add(self.obstacles)
49            self.all_sprites.add(self.level_exit)
50
51        def run(self):
52            while self.running:
53                self.dt = self.clock.tick(60) / 1000.0
54                self.events()
55
56                match self.state:
57                    case "PAUSED":
58                        self.draw_pause_screen()
59                    case "MENU":
60                        self.draw_menu()
61                    case "PLAYING":
62                        self.update()
63                        self.draw()
64                    case "GAMEOVER":
65                        self.draw_game_over()
66
67        def events(self):
68            for event in pygame.event.get():
69                if event.type == pygame.QUIT:
70                    self.running = False
71
72                if event.type == pygame.KEYDOWN:
73                    if event.key == pygame.K_ESCAPE:
74                        if self.state == "PLAYING":
75                            self.state = "PAUSED"
76                        elif self.state == "PAUSED":
77                            self.state = "PLAYING"
78                    else:
79                        self.running = False
80
81                match self.state:
82                    case "PAUSED":
83                        if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
84                            self.state = "PLAYING"
85
86                    case "MENU":
87                        if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
88                            self.new()
89                        self.state = "PLAYING"
90
91                    case "PLAYING":
92                        # Shoot & Reload
93                        if event.type == pygame.KEYDOWN:
94                            if event.key == pygame.K_F and not self.player.is_reloading:
95                                self.player.reload()
96
97            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
98                if self.player.shoot():
99                    mouse_pos = pygame.mouse.get_pos()
100                    new_bullet = projectile.Projectile(self.player.rect.center, mouse_pos)
101                    self.all_sprites.add(new_bullet)
102                    self.bullets.add(new_bullet)
103
104            case "GAMEOVER":
105                if event.type == pygame.KEYDOWN and event.key == pygame.K_F:
106                    self.state = "MENU"
107
108        def update(self):
109            current_time = pygame.time.get_ticks()
110
111            # 1. Spawning
112            if current_time - self.last_spawn_time > self.current_spawn_cooldown:
113                self.spawn_enemy()
114                self.last_spawn_time = current_time
115
116            # 2. Movement & Logic
117            self.all_sprites.update(self.player.rect.center, self.dt, self.obstacles)
118            self.check_collisions()
119
120            # 3. Level Exit Check
121            if self.level_exit and pygame.sprite.collide_rect(self.player, self.level_exit):
122                self.switch_to_level_two()
123
124        def check_collisions(self):
125            # Enemy vs Bullets
126            hits = pygame.sprite.groupcollide(self.enemies, self.bullets, False, True)
127            for enemy in hits:
128                self.score += enemy.value * 10
129
130            # Bullet vs Wall
131            if self.score % 100 == 0 and self.score > 0:
132                self.player.max_health += config.PLAYER_MAX_HEALTH
133                self.player.health = min(self.player.health, self.player.max_health)
134                self.player.max_health = min(self.player.max_health, config.PLAYER_MAX_HEALTH * 1.5)
135
136            # Bullet vs Level Exit
137            if self.score % 500 == 0 and self.score > 0:
138                self.current_enemy_speed = config.ENEMY_SPEED * 1.5
139                self.current_spawn_cooldown = config.SPAWN_COOLDOWN * 0.5
140
141            # Player vs Enemy
142            pygame.sprite.groupcollide(self.player, self.enemies, True, False)
143
144            # Player vs Level Exit
145            if self.level_exit and pygame.sprite.collide_rect(self.player, self.level_exit):
146                self.player.health -= config.PLAYER_MAX_HEALTH * 0.5
147                self.player.health = max(0, self.player.health)
148                self.player.max_health -= config.PLAYER_MAX_HEALTH * 0.5
149                self.player.max_health = max(0, self.player.max_health)
150                self.state = "GAMEOVER"
151
152        def draw_menu(self):
153            """Menu screen"""
154            self.screen.fill(config.BACKGROUND_COLOR)
155
156            self.all_sprites.draw(self.screen)
157
158            instructions = [
159                "WASD to Move",
160                "Space to Shoot",
161                "F to Reload",
162                "Esc to Quit",
163                "R to Restart",
164                "F to Pause",
165            ]
166
167            self.draw_instructions()
168
169        def draw_instructions(self):
170            for i, instruction in enumerate(instructions):
171                text = self.small_font.render(instruction, True, config.TEXT_COLOR)
172                self.screen.blit(text, (50, 100 + i * 30))
173
174        def draw_pause_screen(self):
175            self.screen.fill(config.BACKGROUND_COLOR)
176
177            self.all_sprites.draw(self.screen)
178
179            self.draw_pause_text()
180
181        def draw_pause_text(self):
182            text = self.font.render("PAUSED", True, config.TEXT_COLOR)
183            self.screen.blit(text, (300, 300))
184
185            text = self.font.render("Press F to Resume", True, config.TEXT_COLOR)
186            self.screen.blit(text, (300, 350))
187
188        def draw_game_over(self):
189            self.screen.fill(config.BACKGROUND_COLOR)
190
191            self.all_sprites.draw(self.screen)
192
193            self.draw_game_over_text()
194
195        def draw_game_over_text(self):
196            text = self.font.render("GAME OVER", True, config.TEXT_COLOR)
197            self.screen.blit(text, (300, 300))
198
199            text = self.font.render("Press F to Restart", True, config.TEXT_COLOR)
200            self.screen.blit(text, (300, 350))
201
202            text = self.font.render("Score: {}".format(self.score), True, config.TEXT_COLOR)
203            self.screen.blit(text, (300, 400))

```



```

200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

src > projectile.py > Projectile > update
1 import pygame
2 from . import config
3
4 class Projectile(pygame.sprite.Sprite):
5     def __init__(self, start_pos, target_pos):
6         super().__init__()
7         self.image = pygame.Surface((10, 10), pygame.SRCALPHA)
8         pygame.draw.circle(self.image, config.COLOR2, (5, 5), 5)
9         self.rect = self.image.get_rect(center=start_pos)
10
11         self.pos = pygame.Vector2(start_pos)
12         direction = pygame.Vector2(target_pos) - self.pos
13
14         if direction.length() > 0:
15             self.direction = direction.normalize()
16         else:
17             self.direction = pygame.Vector2(1, 0)
18
19         self.speed = config.BULLET_SPEED
20
21     def update(self, _pos, dt, *args):
22         self.pos += self.direction * self.speed * dt
23         self.rect.center = self.pos
24
25         if not pygame.display.get_surface().get_rect().contains(self.rect):
26             self.kill

```

```
1 import pygame
2 from . import config
3
4 class UManager:
5     def __init__(self):
6         # Fonts loading
7         self.font_main = pygame.font.SysFont("Arial", 48, bold=True)
8         self.font_hud = pygame.font.SysFont("Arial", 22, bold=True)
9
10        self.COLOR_HP = config.COLOR_HP
11        self.COLOR_AMMO = config.COLOR_AMMO
12        self.COLOR_RELOAD = config.COLOR_RELOAD
13        self.COLOR_TEXT = config.COLOR_TEXT
14
15    def draw_hud(self, screen, player, score, elapsed_time):
16        """Zeichnet das HUD während des Spiels"""
17
18        # --- 1. Life (Top Left) ---
19        heart_size = 30
20        for i in range(player.hp):
21            heart_x = 20 + (i * 40)
22            heart_y = 20
23
24            self._draw_heart_shape(screen, heart_x, heart_y, heart_size)
25
26        # --- 2. Score (Below HP) ---
27        score_surf = self.font_hud.render(f"SCORE: {score:04d}", True, self.COLOR_TEXT)
28        screen.blit(score_surf, (20, 45))
29
30        circle_radius = 12
31        padding = 10
32
33        start_x = config.WIDTH - 40
34        start_y = config.HEIGHT - 40
35
36        # --- 3. Ammo (Bottom Right) ---
37        ammo_ratio = player.ammo / player.max_ammo if player.max_ammo > 0 else 0
38        thresholds = [0.25, 0.50, 0.75, 1.0]
39
40        for i in range(4):
41            pos_y = start_y + (i * (circle_radius * 2 + padding))
42
43            pygame.draw.circle(screen, (50, 50, 50), (start_x, pos_y), circle_radius)
44
45            if ammo_ratio > thresholds[i]:
46                color = self.COLOR_AMMO if not player.is_reloading else self.COLOR_RELOAD
47                pygame.draw.circle(screen, color, (start_x, pos_y), circle_radius * 2)
48
49            pygame.draw.circle(screen, self.COLOR_TEXT, (start_x, pos_y), circle_radius * 2)
50
51        ammo_label = self.font_hud.render(f"{player.ammo}", True, self.COLOR_TEXT)
52        screen.blit(ammo_label, (start_x - 10, start_y + 35))
53
54    def draw_screen(self, screen, title, sub_title, bg_color=(0, 0, 150)):
55        overlay = pygame.Surface((config.WIDTH, config.HEIGHT), pygame.SRCALPHA)
56        overlay.fill(bg_color)
57        screen.blit(overlay, (0, 0))
58
59        title_surf = self.font_main.render(title, True, self.COLOR_TEXT)
60        sub_surf = self.font_hud.render(sub_title, True, (255, 255, 255))
61
62        title_rect = title_surf.get_rect(center=(config.WIDTH//2, config.HEIGHT//2 - 40))
63        sub_rect = sub_surf.get_rect(center=(config.WIDTH//2, config.HEIGHT//2 + 40))
64
65        screen.blit(title_surf, title_rect)
66        screen.blit(sub_surf, sub_rect)
```

```
70 def _draw_heart_shape(self, screen, x, y, size):
71     y_adj = y - 8
72
73     circle_radius = int(size * 0.35)
74
75     left_center = (x + circle_radius, y_adj + circle_radius)
76     right_center = (x + size - circle_radius, y_adj + circle_radius)
77     pygame.draw.circle(screen, self.COLOR_HP, left_center, circle_radius)
78     pygame.draw.circle(screen, self.COLOR_HP, right_center, circle_radius)
79
80     triangle_points = [
81         (x, y_adj + circle_radius),
82         (x + size, y_adj + circle_radius),
83         (x + size // 2, y_adj + size)
84     ]
85     pygame.draw.polygon(screen, self.COLOR_HP, triangle_points)
```

9 Anhang Unit/Gameplay Tests

```
1 import pytest
2 import pygame
3 from src.enemy import Enemy
4 from src import config
5
6 @pytest.fixture(autouse=True)
7 def setup_pygame():
8     pygame.init()
9     pygame.display.set_mode((1, 1), pygame.NOFRAME)
10    yield
11    pygame.quit()
12
13 @pytest.fixture
14 def enemy():
15     return Enemy(100, 100, 2)
16
17 # --- TEST DMG-LOGIC ---
18
19 def test_enemy_take_hit(enemy):
20     initial_hp = enemy.hp
21     score = enemy.take_hit(1)
22     assert enemy.hp == initial_hp - 1
23     assert score == config.ENERGY_SCORE_VALUE
24
25 def test_enemy_death(enemy):
26     group = pygame.sprite.Group()
27     group.add(enemy)
28
29     enemy.take_hit(enemy.hp)
30     assert enemy.hp <= 0
31     assert enemy not in group
32
33 # --- TEST KILL MOVEMENT ---
34
35 def test_enemy_movement_towards_player(enemy):
36     initial_pos = pygame.Vector2(enemy.pos)
37     # Spieler ist rechts vom Gegner (200, 100)
38     player_pos = (200, 100)
39     dt = 1.0
40     obstacles = []
41
42     enemy.update(player_pos, dt, obstacles)
43
44     assert enemy.pos.x > initial_pos.x
45     assert enemy.pos.y == initial_pos.y
46
47 def test_enemy_hitbox_follows_pos(enemy):
48     player_pos = (200, 200)
49     enemy.update(player_pos, 1.0, [])
50
51     assert enemy.hitbox.centerx == round(enemy.pos.x)
52     assert enemy.hitbox.centery == round(enemy.pos.y)
```

```
tests > test_player.py > ...
1 import pytest
2 import pygame
3 from src.player import Player
4
5 # Mini Setup
6 @pytest.fixture(autouse=True)
7 def setup_pygame():
8     pygame.init()
9     pygame.display.set_mode((1, 1), pygame.NOFRAME)
10    yield
11    pygame.quit()
12
13 @pytest.fixture
14 def player():
15     return Player()
16
17 # --- TEST FOR TAKING DMG ---
18
19 def test_take_damage(player):
20     initial_hp = player.hp
21     player.take_damage(1)
22     assert player.hp == initial_hp - 1
23     assert player.is_alive is True
24
25 def test_player_death(player):
26     player.take_damage(player.hp)
27     assert player.hp == 0
28     assert player.is_alive is False
29
30 def test_hp_not_negative(player):
31     player.take_damage(999)
32     assert player.hp == 0
33
34 # --- TEST FOR SHOOTING ---
35
36 def test_shoot_reduces_ammo(player):
37     initial_ammo = player.ammo
38     success = player.shoot()
39     assert success is True
40     assert player.ammo == initial_ammo - 1
41
42 def test_shoot_fails_when_empty(player):
43     player.ammo = 0
44     success = player.shoot()
45     assert success is False
46     assert player.ammo == 0
47
48 def test_cannot_shoot_while_reloading(player):
49     player.reload()
50     success = player.shoot()
51     assert player.is_reloading is True
52     assert success is False
```

```
tests > test_projectile.py > ...
1 import pytest
2 import pygame
3 from src.projectile import Projectile
4 from src import config
5
6 @pytest.fixture(autouse=True)
7 def setup_pygame():
8     pygame.init()
9     pygame.display.set_mode((800, 600), pygame.NOFRAME) # Screen-Größe für Boundary-Check
10    yield
11    pygame.quit()
12
13 def test_projectile_initialization():
14     start = (100, 100)
15     target = (200, 100)
16     bullet = Projectile(start, target)
17
18     assert bullet.direction.x == 1.0
19     assert bullet.direction.y == 0.0
20
21 def test_projectile_movement():
22     start = pygame.Vector2(100, 100)
23     target = (200, 100)
24     bullet = Projectile(start, target)
25
26     dt = 1.0
27
28     bullet.update(None, dt)
29
30     expected_x = start.x + config.BULLET_SPEED
31
32     assert bullet.pos.x == expected_x
33     assert bullet.rect.centerx == round(expected_x)
34
35 def test_projectile_zero_direction():
36     bullet = Projectile((100, 100), (100, 100))
37     assert bullet.direction.length() == 1.0
```

10 Anhang README.md

306 SCB - Top-Down Shooter

Ein intensiver Survival-Shooter, entwickelt im Rahmen des Moduls 306 mit Python und Pygame-CE. Überlebe so lange wie möglich gegen feindliche Wellen und maximiere deinen Highscore!

Installation & Start (Windows)

Das Spiel wird als eigenständige Applikation geliefert und benötigt keine installierte Python-Umgebung.

1. Navigieren Sie/ Laden Sie die den Ordner Herunter in den Ordner `game`.
2. Starten Sie die Datei `TopDownShooter.exe`.
3. Wichtiger Hinweis: Da die Datei nicht digital signiert ist, erscheint beim Start möglicherweise der Windows-SmartScreen. Klicken Sie auf "Weitere Informationen" -> "Trotzdem ausführen".

Bedienung

Taste	Aktion
W, A, S, D	Bewegung des Charakters
Maus	Zielen
Linksklick	Schießen
R	Nachladen (Manuell oder automatisch)
ESC	Spiel pausieren / Beenden (im Hauptmenü)
ENTER	Spiel starten (Menü / Pause)

Hauptmerkmale (Features)

- Zwei Level-Umgebungen: Von einem Einführungslevel bis hin zu komplexen Arealen mit Sackgassen und taktischen Engpässen.
- Dynamische Skalierung: Der Schwierigkeitsgrad (Geschwindigkeit und Spawn-Rate) passt sich kontinuierlich dem Score an.
- Upgrade-Mechanik: Alle 100 Punkte erfolgt eine automatische Magazinerweiterung (+5 Schuss).
- Flüssige Steuerung: Einsatz von "Sliding Collisions" für eine barrierefreie Bewegung an Hindernissen.
- Benutzerschnittstelle: Echtzeit-HUD und ein aktives Benachrichtigungssystem für Upgrades und Warnungen.

Technische Umsetzung (EVA-Prinzip)

Die Software-Architektur ist nach dem EVA-Prinzip strukturiert:

1. Eingabe: Erfassung von Benutzersignalen über eine zentrale State-Machine.
2. Verarbeitung: Vektor-basierte Bewegungslogik und präzise Kollisionsabfragen auf zwei Achsen.
3. Ausgabe: Sprite-basiertes Rendering mit stabilen 60 FPS und Overlay-UI.

Projektteam

Entwickelt im Rahmen der Informatikausbildung (Modul 306). Entwickler: Stefanie Gerber, Benjamin Phengrasamy, Sujanthan Suntheralingam, Christophe Grädel